

AI-Driven Vertical Farming: An Intelligent System For Optimized Growth



Final Year Project Report

Presented

by

Ahsan Raza Shah

CIIT/FA22-BCE-018/ISB

Arooba Wasim

CIIT/FA22-BCE-025/ISB

Misbha Arif

CIIT/FA22-BCE-046/ISB

In Partial Fulfillment

of the Requirement for the Degree of

Bachelor of Science in Computer Engineering

DEPARTMENT OF COMPUTER ENGINEERING

COMSATS UNIVERSITY ISLAMABAD

May 2026

AI-Driven Vertical Farming: An Intelligent System For Optimized Growth



Final Year Project Report

Presented

by

Ahsan Raza Shah

CIIT/FA22-BCE-018/ISB

Arooba Wasim

CIIT/FA22-BCE-025/ISB

Misbha Arif

CIIT/FA22-BCE-046/ISB

In Partial Fulfillment

of the Requirement for the Degree of

Bachelor of Science in Computer Engineering

DEPARTMENT OF COMPUTER ENGINEERING

COMSATS UNIVERSITY ISLAMABAD

May 2026

Declaration

We, hereby declare that this project neither as a whole nor as a part there of has been copied out from any source. It is further declared that we have developed this project and the accompanied report entirely on the basis of our personal efforts made under the sincere guidance of our supervisor. No portion of the work presented in this report has been submitted in the support of any other degree or qualification of this or any other University or Institute of learning, if found we shall stand responsible.

Signature: _____

Name: Ahsan Raza Shah

Signature: _____

Name : Arooba Wasim

Signature: _____

Name: Misbha Arif

COMSATS UNIVERSITY ISLAMABAD

May 2026

AI-Driven Vertical Farming: An Intelligent System For Optimized Growth

An Undergraduate Final Year Project Report submitted to the
Department of
COMPUTER ENGINEERING

As a Partial Fulfillment for the award of Degree
Bachelor of Science in Computer Engineering
by

Name	Registration Number
Ahsan Raza Shah	CIIT/FA22-BCE-018/ISB
Arooba Wasim	CIIT/FA22-BCE-025/ISB
Misbha Arif	CIIT/FA22-BCE-046/ISB

Supervised by

Dr. Mohsin Riaz
Associate Professor

Co Supervised by
Mr. Tayyab Rasul
Lecturer

Department of Electrical and Computer Engineering
CU Islamabad

COMSATS UNIVERSITY ISLAMABAD

May 2026

Final Approval

This Project Titled
AI-Driven Vertical Farming: An Intelligent System For Optimized Growth

Submitted for the Degree of Bachelors of
Science in Computer Engineering

by

Name

Ahsan Raza Shah

Registration Number

CIIT/FA22-BCE-018/ISB

Arooba Wasim

CIIT/FA22-BCE-025/ISB

Misbha Arif

CIIT/FA22-BCE-046/ISB

has been approved for

COMSATS UNIVERSITY ISLAMABAD



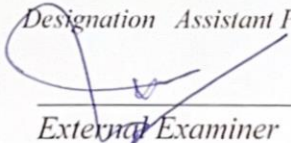
Supervisor Dr. Mohsin Riaz
Professor



Internal Examiner-I
Name, Dr. Babar Mansoor
Designation Assistant Professor



Internal Examiner-II
Name, Khurram Ali
Designation Assistant Professor



External Examiner
Name, Dr. Waqas Haider Bangyal
Designation Associate Professor

Head
Department of Computer Engineering

Dedication

We are starting with the name of ALLAH, who has strengthened us throughout this journey. This thesis is first dedicated to ALLAH, who has shown us the right path.

This thesis is dedicated to all those people who have remained our constant support throughout our 4years journey, including our project supervisor, teachers, and our parents. All these people encourage us to learn from our mistakes and help us to become a better version of us in both professional and personal life.

Acknowledgements

We look at Allah Almighty to whom all the Gratitude belongs for the opportunity that he gave us to study at COMSATS University. Support frames in our engineering journey to award our final year project certainly like many. We are particularly grateful to our supervisor, Dr. Mohsin Riaz and Co supervisor, Sir Tayyab Rasul not only for providing their advice, but also believe so that contributing their knowledge and time is not a way of reward. They were just as fully aware of our challenges as we were, and they offered the best ways in which we could solve our problems, which was key for the project. We never fought with the situations of our life, because we had parents, who caught us and let us work with the ease. We wouldn't have been able to complete the project without the support of the people in the Computer Engineering Department, college. Last but not least, thanks to COMSATS University where we have been studying for education and the important chances that made such a project possible.

Ahsan Raza Shah

Arooba Wasim

Misbha Arif

Table of contents

Chapter 1	0
Introduction	0
1.1 Problem Statement	1
1.2 Purpose and Objectives	1
1.3 Project Overview	2
1.4 Expected Outcomes	3
❑ Functional Prototype	3
❑ Intelligent Decision-Making System	3
❑ Cost-Effective Solution	4
❑ Real-Time Monitoring Dashboard	4
1.5 Applications	4
1.6 Target SDGs (Sustainable Development Goals)	5
1.6.1 SDG-2: Zero Hunger	5
1.6.2 SDG-9: Industry, Innovation and Infrastructure	5
1.6.3 SDG-12: Responsible Consumption and Production	5
1.6.4 SDG-13: Climate Action	5
Chapter-2	6
Literature Review.....	6
2.2 Research Gap	6
2.3 Vertical Farming and Controlled Environment Agriculture	7
2.3.1 Existing Commercial Solutions	7
2.4 Critical Analysis of Existing Solutions	8
2.4.1 Basic IoT Monitoring Systems	8
2.4.2 Machine Learning Systems	8
2.4.3 Computer Vision Plant Disease Systems	9
2.5 Environmental Sensing and Data Communication	9
2.5.1 Sensor Types for Indoor Farming.....	10
2.5.2 Communication Protocols for Embedded Systems	12
2.6 Machine Learning for Tabular Data	14
2.6.1 Classical Approaches	14

2.6.2 Deep Learning on Tabular Data	14
2.6.3. TabNet.....	15
2.7 Computer Vision for Plant Health Diagnosis	15
2.7.1 The PlantVillage Dataset	16
2.7.3 Self-Supervised Learning and SimCLR	17
2.7.4 K-Means Clustering for Pseudo-Labeling	17
2.7.5 Lightweight CNN Architectures for Edge Deployment	19
2.8 Hardware Analysis	20
2.8.1 Which Microcontroller to Choose?	20
2.8.2 Central Processing Unit: Raspberry Pi 5	22
2.8.3 Cloud Platform: Firebase	22
2.9 Development Environment	23
2.9.1 Arduino IDE	23
2.9.2 Python on Raspberry Pi 5	23
2.9.3 Machine Learning Training Environment	24
Chapter-3	25
Methodology	25
3.1 Hardware Architecture	25
3.1.1 Hardware Components	25
3.1.2 System Overview	32
3.1.3 Hardware Block Diagram	33
3.1.4 Flowchart	34
3.2 Software Architecture	34
3.2.1 ESP32 Firmware	36
3.2.2 Central Coordination	37
3.2.3 UART Communication and Actuator Feedback Loop	38
3.2.4 TabNet Environmental Prediction Pipeline	38
3.2.5 Visual Plant Health Diagnosis Pipeline	41
3.2.6 Cloud Integration and Monitoring Ecosystem	46
3.2.7 Design Considerations	47
Chapter-4	49

Results and Discussions	49
4.1 Introduction	49
4.2 Clustering Results	49
4.2.1 Overview	49
4.2.2 Clustering Evaluation Matrices	50
4.2.3 Distribution Of images	51
4.3 Dataset Balancing.....	52
4.3.1 Motivation for Balancing.....	52
4.3.2 Method and Result of Balancing	52
4.4 CNN training and evaluation outcomes	54
4.4.1 Training Results	54
4.4.2 Final Test Set Performance	54
4.4.3 Per-Class Analysis	55
4.4.4 Confusion Matrix Analysis	56
4.4.5 Gap between Validation and Test Accuracy.....	58
4.5 Overall System Evaluation	58
4.5.1 End-to-End Pipeline Assessment	58
4.5.2 Significance of Results in Context	59
4.5.3 Hardware Deployment Performance	59
4.6 Summary	60
Chapter-5	60
Conclusion and Future Work	60
5.1 Project Summary	61
5.2 Achievement of Objectives	62
5.3 Key Contributions	63
5.4 Limitations	64
5.5 Future Work	66
5.5.1 Growth-Stage-Specific Environmental Recommendations	66
5.5.2 Expanded and Verified Plant Health Dataset	66
5.5.3 Wireless Communication for Scalability	66
5.5.4 Dedicated CO2 Monitoring	67
5.5.5 Reinforcement Learning for Closed-Loop Optimisation	67
5.5.6 Multi-Crop Simultaneous Management	67
5.5.7 Integration of a Nutrient Dosing System	67

5.5.8 Yield Prediction and Harvest Optimisation	68
5.6 Reflections on the Project	68
5.7 Conclusion	69

List of Acronyms

AI	Artificial Intelligence
ADC	Analog-to-Digital Converter
API	Application Programming Interface
AWS	Amazon Web Services
BH1750	Digital Light Intensity Sensor Module
CEA	Controlled Environment Agriculture
CNN	Convolutional Neural Network
CO ₂	Carbon Dioxide
DHT22	Digital Temperature and Humidity Sensor
ESP32	Espressif 32-bit Microcontroller
GPIO	General Purpose Input/Output
GPU	Graphics Processing Unit
HSL	Hue, Saturation, Lightness
I ² C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
K-Means	K-Means Clustering Algorithm
LED	Light Emitting Diode
ML	Machine Learning
MQ-135	Air Quality Gas Sensor
NPK	Nitrogen, Phosphorus, Potassium
NT-Xent	Normalized Temperature-scaled Cross-Entropy Loss
Pi	Raspberry Pi
RAM	Random Access Memory
REST	Representational State Transfer
RS-485	Recommended Standard 485 (Serial Communication Protocol)
SDG	Sustainable Development Goal
SimCLR	Simple Framework for Contrastive Learning of Visual Representations
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VGG	Visual Geometry Group (CNN Architecture)
Wi-Fi	Wireless Fidelity

List of Figures

Figure 1: CIRCUIT DIAGRAM	13
Figure 2: Methodology diagram.....	26
Figure 3: ESP32 MICROCONTROLLER	27
Figure 4:ESP32 Pinout Diagram	28
Figure 5:NPK 7-in -1 SOIL SENSOR.....	28
Figure 6:DHT22 SENSOR	29
Figure 7: MQ-135 AIR QUALITY	30
Figure 8:BH1750 LIGHT INTENSITY SENSOR	30
Figure 9: WATER LEVEL SENSOR	31
Figure 10:ACTUATORS; Heater , Fan ,Pump ,LED	31
Figure 11: Raspberry Pi5	32
Figure 12: Camera module	33
Figure 13: Hardware Block Diagram	34
Figure 14: Flowchart	35
Figure 15: Software Flowchart	37
Figure 16:Clustering Evaluation Matrices	51
Figure 17: Distribution Of Images Across Clusters	52
Figure 18:Final dataset distribution after balancing	54
Figure 19:Test Split of the Balanced Dataset	54
Figure 20:Per-class classification report	56
Figure 21:Confusion matrix for the MobileNetV3-Small classifier	58

List of Tables

Table 1: Comparison of existing solution categories with this project	10
Table 2: Protocols and their role	14
Table 3: K-Means Clustering for Pseudo-Labeling	19
Table 4: CNN architecture comparison for Raspberry Pi edge deployment	21
Table 5: Microcontroller comparison for tray-level sensing role	22
Table 6: Cloud platform comparison	24
Table 7: Four-layer architecture summary	36
Table 8: TabNet hyperparameter configuration	40
Table 9: Multi-head MobileNetV3-Small output head architecture	44
Table 10: MobileNetV3-Small fine-tuning training configuration	45
Table 11: Firebase Realtime Database path schema	47

Abstract

Growing smarter and sustainable food production systems has become even more critical due to several reasons like climate change, increasing urban population, and loss of farmland that threatens the sustainability of traditional farming methods. Vertical farms offer a promising in-door farming solution and are replacing the traditional farms. However, the majority of existing vertical farms systems rely on manual supervision or, rigid systems with simple rules based on system reliance that cannot cater to varying requirements of crops. This project presents the engineering and design of a smart farming system, based on the combination of IoT devices analytics, artificial intelligence, and software feedback control systems to address these issues and limitations.

The farming system consists of two modular farming trays that operate as individual microfarms , each controlled by an ESP32 networked microcontrollers. Each microfarm collects real-time values of their operating environment from sensors (temperature, humidity, soil moisture, pH levels, light, air quality, nitrogen, potassium and phosphorus). ESP32 microcontrollers are used to create a feedback control system that analyzes the values it collects from sensors to automatically respond to changing conditions. A smart farming system based on control functions creates and self-adjusts problems via feedbacks. Sensor data will sent through an ESP32 to Raspberry Pi5 and integrated with Firebase (cloud-based production) to process real time monitoring and autonomous control of the microfarms.

The system employs TabNet on the intelligence side. TabNet is useful for deep learning which deals tabular data efficiently. It is used to predict optimal environmental conditions for a given crop and issue appropriate actuator commands to ESP32 that manage fans, pumps, and lights. For plant health diagnostics, a self-supervised visual pipeline using SimCLR was developed to derive useful representations from around 10,000 unlabeled leaf photos from the leaf image data found in the

“PlantVillage” dataset. K-Means clustering organized the unlabeled images into seven distinct health categories. They are used to fine-tune the MobileNetV3 CNN which was directly deployed on the Raspberry Pi for the purpose of real-time, CPU-based leaf diagnosis.

The system functions as an automated solution where sensor intelligence, predictive modeling and visual diagnostics integrate to keep crops healthy with minimal human intervention.

Chapter 1

Introduction

Food security for an increasing population presents a growing challenge for society. The United Nations estimates the world's population will be 9.7 billion by 2050 [27]. This will require global food production to be 70% greater. Climate change, urbanization, and decreasing fresh water exacerbates the problem for an already predictable growing season. The only solution for food security and growing population is conventional agriculture.

Vertical farming is a great alternative to traditional farming. Thickening the landscape by growing farms upwards in a controlled indoor environment rather than horizontally. Vertical farming eliminates the dependency on weather, farming footprint is heavily reduced, and water use is recycled. This allows farms to grow crops in places that is traditionally impossible to grow crops like rooftops, repurposed warehouses, and even arid desert regions.

Vertical farming invokes the use of technology to increase the efficiency of farming, which is especially useful as population increases, though it does have some drawbacks. One of the biggest drawbacks is the requirement of a large amount of energy to sustain the technology used. The farming technology requires the use of artificial lighting, heating, and cooling, which costs large amounts of energy. Essentially, the farming technology is too expensive for the benefits to justify the price. Another drawback involves the constant and exceedingly intricate management of the farm's environment. The technology used to grow the crops needs to maintain large ranges for temperature, humidity, CO₂, and light. However, the needs of crops are variable, and the technology is not able to adapt as needed. The technology is not able to adjust the needs of the crops quickly, and the human farmer does not have the ability to quickly respond to the constant fluctuation.

In conjunction with the farming technology, artificial intelligence provides the biggest benefits of vertical farming. AI is immensely helpful especially when farming the technology is combined with the AI technology of machine learning and the study of agronomy. AI provides the benefits

of predicting the needs of the crops before the crops reach a point of deficiency, and AI is useful when identifying malformation in crops that a human may overlook [25], [26].

1.1 Problem Statement

Today, the majority of commercial vertical farm operations fall into two classes. First are manual farms such as Spruce & Co. where farmers are responsible for overseeing the external conditions through the use of various sensors and making manual adjustments. This type of farm is labor intensive and is subject to delays. Second are farms that implement some type of automation, where that automation is the implementation of rules. For instance, the fans and irrigation systems turn on and off when the temperature or soil moisture are elevated or reduced, respectively. This type of automation is much better than manual farms, but it is still reactive as opposed to predictive. Additionally, it is unable to consider the needs of a crop, therefore, it is unable to operate at the appropriate crop stage.

The gap in the market which this project aims to target, is the use of accurately priced and widely available embedded hardware paired with real machine learning on the edge. This allows the farm to not just monitor the state of the farm, but to also predict the state in which it should operate, and also diagnose the health of the crop in an integrated manner.

1.2 Purpose and Objectives

The main goal of this project is to develop an AI powered vertical farming, where crop growth is managed through a sensor network, a cloud data system, and two integrated machine learning models. To achieve this goal, the following objectives are established:

- To build a functional, smart, integrated, hierarchically structured system that utilizes ESP32 Microcontrollers and a Raspberry Pi 5 in order to collect sensor data and distribute and execute realtime AI tasks.

- Our goal is to create a data pipeline, where the sensor readings will be stored temporarily, processed, and then sent to Firebase, where readings will be displayed on a monitoring dashboard.
- Our goal is to train and test a TabNet model to perform environmental forecasting using the Crop Recommendation Dataset[1].
- To employ a SimCLR self-supervised feature extraction approach followed by K-Means clustering, and finally train a MobileNetV3 CNN to construct a plant health classification model using the PlantVillage Dataset[3],[10the].
- To consolidate the hardware components and the software modules to create a prototype and analyze the system.

1.3 Project Overview

This project aims to create a standalone vertical farming system embedded with Artificial Intelligence algorithms to monitor and optimize the health and growth conditions of plants. The system captures the hardware and the software to create a complete working prototype that gathers data and makes data-driven and rational decisions. The prototype captures a data and decisionmaking cycle using RL (reinforcement learning) methodology and grows plants simultaneously.

Traditionally, farming methods relied on the farmer to check the health of the plants and assign their needs. The proposed system eliminates this by implementing observant automation where growth conditions can be adjusted iteratively to create the “ideal” state” for the plants based on data achieved and decisions taken in the cycle in addition to the automation with learning algorithms.

This AI-Driven Vertical Farming System project is an end-to-end project from hardware design, embedded systems, cloud, and machine learning. At its physical core, the system comprises of a two tray vertical growing system. Each tray has an ESP32 and a collection of environmental sensors to measure temperature, humidity, soil moisture, CO2 concentration, and light intensity.

All these sensors are then connected to each ESP32 and send sensors information in real-time to Raspberry pi through ESP32. The ESP32 of both trays separately send sensor values of the growing system to the Raspberry Pi 5 (RPi 5) through a UART connection.

In this system, the Raspberry Pi 5 serves as the main controller. It receives and processes the sensor data, uploads the data to Firebase for display on a dashboard, and runs the TabNet model. The TabNet model is used to provide environmental control recommendations. The Raspberry Pi 5 also runs the MobileNetV3 model to perform analysis for leaf images captured using a camera module. Based on the TabNet's result, the Raspberry Pi 5 sends commands to the ESP32s. These commands then control different system actuators (e.g., fans, growing lights, and water pumps) according to environmental control recommendations. Based on the MobileNetV3's result, the Raspberry Pi 5 updates the dashboard with the detected plant health issues for the selected plant. The plant will be selected through dashboard.

Both TabNet and MobileNetV3 were developed and trained using offline dedicated datasets. The TabNet model was trained on the Crop Recommendation Dataset, which is publicly available on Kaggle. The MobileNetV3 model was trained using a labeled dataset created through a selfsupervised scheme utilizing the Plant Village Dataset, which features a collection of plant and crop images, and then predicting clusters using K-means clustering. After training, TabNet and MobileNetV3 were optimized for the Raspberry Pi (RPi) to run on-device inference..

1.4 Expected Outcomes

The project aims to achieve the following outcomes:

Functional Prototype

A working prototype of the vertical farming system will be developed, demonstrating real-time monitoring, data processing, and automated decision-making for plant health.

Intelligent Decision-Making System

The system should be able to analyze sensor data and suggest or apply optimal conditions for plant growth using machine learning.

Cost-Effective Solution

One of the main goals is to keep the system affordable so that it can be used by students, smallscale farmers, and hobbyists and can be placed or kept anywhere like in homes, shops and kitchens etc.

Real-Time Monitoring Dashboard

A user-friendly dashboard will allow users to view live data and system outputs, making the system easy to use and understand.

1.5 Applications

However, since the application of this project focuses on vertical farms as primary applications and as one form of agriculture. It is the application of this project in agriculture that covers more and wider regions. The project application will be more appropriate and suitable for planting in Urban Farming where we lack land and limited space. The project enables users to grow plants indoors for residential use, apartments, commercial spaces, rooftop plantings, etc.

The plants' required environment is monitored with automated sensors to provide a balance between plants and the ambient environment within an enclosed space. Applications in Indoor Plant Monitoring Systems with temperature, humidity, soil moisture, light intensity monitoring features, the product is the same as a plant monitor. You can also use it in indoor farming, and Greenhouse applications with better management and monitoring capabilities. Applications in agricultural laboratories for experimental farming in agricultural research centers, with the help of the smart irrigation system, a controlled environment can be built.

These experiments can help learn and develop farming techniques, plant studies and the impact of plants with the ambient. For use in projects, educational and exhibition purposes Since agriculture in smart agriculture, it uses sensors, IoTs, and Automation technologies, so you can create and use an amazing and illustrative model. You will learn with smart farming applications. In future disease

detection using image analysis, auto nutrient and water supply, the artificial intelligent based monitoring of the plants, remote accessing through the mobile app, Large-scale Smart farming.

1.6 Target SDGs (Sustainable Development Goals)

This project has positive effects on many of the United Nations Sustainable Development Goals.

1.6.1 SDG-2: Zero Hunger

The strongest correlation is with SDG-2 and its objective for Sustainable Zero Hunger. This project advances the intelligent and sustainable resource efficiency of vertical farming systems. It helps close the sustainable food production barrier that is lacking in areas unsuitable for traditional farming systems.

1.6.2 SDG-9: Industry, Innovation and Infrastructure

The integration of AI, IoT sensors, and Cloud in the farming environment is precisely the kind of innovation in emerging technologies that SDG-9 is promoting. The project shows how easily accessible hardware and farming systems based on ESP32, Raspberry Pi, and open-source ML platforms can easily build systems.

1.6.3 SDG-12: Responsible Consumption and Production

Compared to traditional farming, vertical farming consumes less water. The integration of AI within this project further minimizes waste, as it allocates resources (water, light, and nutrients) as needed and not on a schedule. This practice supports SDG-12 and promotes responsible and efficient production practices.

1.6.4 SDG-13: Climate Action

This project helps food production ways that allow independence from climate conditions. This supports climate adaptation to communities that experience extreme weather and shifting seasons.

Chapter-2

Literature Review

With the population increasing at an alarming rate every year, allied food production has to keep pace with the spike of new consumers. Beyond population growth, food producers must adapt to changing farmland conditions, including land degradation and shifting terrain, while also dealing with unpredictable climate changes that negatively impact food production. The answer to some of these compounded problems is vertical farming[25]. Once the vertical systems are in place, growing layers of crops can be adjusted as necessary. The environmental controls can be kept at a year round steady equilibrium. The main hindrance to the design of these farming systems is the manual oversight of the farming systems themselves. Utilizing AI is a great avenue to replace more basic systems of dependence by allowing each crop to be proceeded and adjusted to its individual needs[25],[26]. This paper proposes new ways to implement over these types of systems. There are also new ways to offset the control of farming systems biproducts such as, cloud servers, distributed ESP32, and predictive analytic softwares that together can work in place of farming system modules.

2.2 Research Gap

During our research into vertical farming automation, IoT-based agricultural sensor networks, machine learning for crop recommendation, and computer vision for plant disease detection, we found a consistent and important gap in the existing literature[26]. Most recent studies have only addressed specific elements within this concern, for instance, dash boards for monitoring with no incorporation of AI, training a software machine learning model that runs independently of the hardware, or even the development of a classifier that is not tested in a farming field[25],[26].

The study addresses three components, intelligent control of the environment with TabNet, which is a deep learning model tailored for tabular data of sensors, visual health of plants and diagnostics by means of a self-supervised feature extraction and clustering process, and the control of edgedeployed MobileNetV3[3], and multi-sensing distributed ESP32 microcontrollers on each farming tray with individual and direct communication to a Raspberry Pi 5 over UART[1]. To our knowledge no other study has deployed an integrated vertically controlled intelligent farming assistant (virtual and physical AI) in the literature. This study aims to bridge this gap.

2.3 Vertical Farming and Controlled Environment Agriculture

Controlled environment agriculture (CEA) encompasses all approaches to creating a controlled growing environment for the crops[25]. The simplest version of the execution of this idea could be the use of a greenhouse, vertical farming takes a step beyond this concept with the omission of sunlight. This is substituted with artificial horticultural lighting, with the vertical stacking of growing layers.

Vertical farming's agronomy base built on artificial light spectra and photosynthesis, growing crops in water and control of temperature and humidity, was achieved through lab research in the 1980s and early 1990s. This work set the boundaries for current vertical farm environmental systems.

In recent years, the emphasis has switched to automation and system self-management[26]. The advancement of vertical farming has highlighted the shortcomings of system management based on manual and procedural controls. Farmers using manual control systems face inconsistent harvest as the lag between discovering and addressing farm system failures results in damaged crops. Automated systems that utilize simple thresholds for control are better, but still cannot predict whether external or internal system failures are about to exceed a threshold. The multidimensional nature of control systems require prediction, control and adjustment of system variables to achieve balance, at different phases of farm crop systems. They require selfmanagement.

2.3.1 Existing Commercial Solutions

Commercial vertical farming companies like AeroFarms, Plenty, and Bowery Farming have proprietary automated systems, but their technical implementations have not appeared in the public domain. What is known is these systems depend on large-scale, capital-intensive infrastructure, thus they are not adaptable to research or small-scale deployment systems. Our work demonstrates that the same or similar level of system intelligence can be implemented in intelligent vertical farming systems at research and small companies, and not just at large commercial companies, using inexpensive, off-the-shelf hardware, along with open-source machine learning systems.

2.4 Critical Analysis of Existing Solutions

Based on the current state of the art, existing systems address the not fully automated vertical farming systems, but none integrate and deploy fully automated systems. All existing systems fall into one of three categories.

2.4.1 Basic IoT Monitoring Systems

The basic IoT monitoring system is the most recurrent instance in the agricultural engineering literature. The most frequent microcontrollers in use are devices based on ESP32 or Arduino. Sensor readings are sent via the microcontroller to the cloud, where results are visualized in a dashboard. In the documented instances, there is no machine learning involved[16],[19]. In the instances where actuator control is documented, the system would activate a pump or fan based on a hard-coded rule, such as a sudden fall in soil moisture, or a sudden rise in temperature. Remote monitoring systems of this type, while useful, are limited to the current moisture or temperature conditions, without the ability to predict the automated control conditions based on the type of crop, or assist in determining the visual condition of the plant.

2.4.2 Machine Learning Systems

The next category includes crop recommendation or environmental control machine learning models that use software only simulation without real hardware systems. These models convince peer reviewers by showing that AI models can predict crop environmental conditions using various benchmark datasets. These papers, however, do not take that final step where crop specific models are practically deployed and integrated with real farms to control real actuators using sensor feedback. The tendency to evaluate models but not deploy them is prevalent in most of the Agricultural AI literature.

2.4.3 Computer Vision Plant Disease Systems

The final category includes standalone computer vision systems designed to classify the diseases of plants using leaf images. The common dataset used is PlantVillage. These models report accuracies greater than 95% on the test sets from that dataset, however, the models are not deployed on resourceconstrained edge devices, and they are not integrated into real-time monitoring systems. The PlantVillage dataset was not constructed using a self-supervised scheme from unlabeled images.

Table 2.1: Comparison of existing solution categories with this project

Solution Type	Env. Prediction	Visual Diagnosis	Hardware Integration	Real-Time Deployment
Basic IoT Monitoring	No	No	Yes	Partial
ML Crop Recommendation (Standalone)	Yes	No	No	No
CNN Disease Detection (Standalone)	No	Yes	No	No
This Project	Yes (TabNet)	Yes (MobileNetV3)	Yes (ESP32 + Pi 5)	Yes

2.5 Environmental Sensing and Data Communication

The system depends on effective sensing of its environment and the capacity to seamlessly communicate that data to manage the environmental parameters critical for the healthy growth of the plants. In indoor agriculture, any plants grown inside will have no environmental support, as any environmental parameters the plants depend on will be wholly reliant on an artificially controlled environment. It is critical for plant health and maximum yield of crops in indoor agriculture for us to constantly measure certain aspects of the environment, such as air quality, temperature, humidity, air

humidity, light intensity, and nutrient concentrations of the nutrient solution. Sensors and communication modules are embedded in the system for acquiring environmental data, processing this data, and communicating it to a centralized location[26].

This data acquisition mechanism monitors the surrounding environment of the plants and will facilitate smart decisions in watering the plants, the intensity of the light, the air supply, and nutrient distribution in the growing medium. Also, communication protocol helps us in smoothly exchanging of this sensed information between the different components of the smart system (i. e sensors, microcontroller, processing and the cloud environment). This part deals with various types of sensors for collecting different parameters and the communication protocols that facilitate the communication of this collected data.

2.5.1 Sensor Types for Indoor Farming

The appropriate management of the growing environment in each individual vertical farm unit requires continuous monitoring of the optimal growing conditions' parameters. The relevant review of the agronomic and agricultural engineering literature identified the critical agronomic and engineering parameters for indoors farm operations as the temperature and relative humidity, soil moisture, concentration of CO₂, intensity of illumination, and levels of nutrients in soil in potassium, phosphorus and nitrogen.

DHT22 - Temperature and Humidity: The trays are equipped with DHT22 temperature and humidity sensors. The technology in DHT22 is based on a digital one-wire protocol. The accuracy of temperature measurements is $\pm 0.5^{\circ}\text{C}$, and for relative humidity, it is in the range of $\pm 2\text{-}5\%$. The DHT11 temperature and humidity sensor was excluded as the DHT22 provides optimal accuracy in the case of the small, critical growing condition deviations that the TabNet model identifies[23].

NPK 7-in-1 Soil Sensor Conductivity: One of the tray has one NPK 7-in-1 sensor. This sensor determines the concentration of soil N, P and K, pH, volumetric water content, temperature and soil conductivity. The NPK sensor has a RS-485 communication that uses the Modbus RTU protocol, and the ESP32 via a UART interface RS-485 transceiver also uses this same protocol.

MQ-135 -Air Quality:

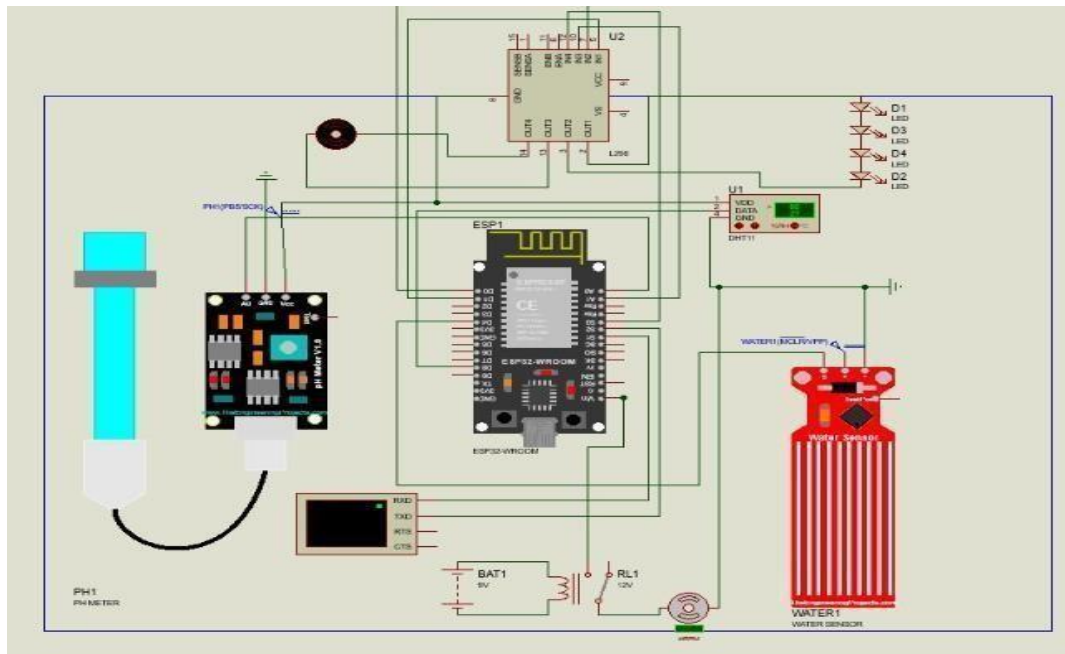
The MQ-135 gas sensor is integrated to detect the general air quality inside the enclosed vertical farming space[24]. Poor ventilation or the accumulation of hazardous gases in a confined farming system over time may eventually decay plant growth; thus, it becomes necessary to continuously monitor the air quality to prevent potential hazards. As such, the MQ-135 generates an analog signal output that can be processed using an ESP32's 12-bit ADC with a calibrated offset to compute an air quality index. It should be emphasized that despite the broad gas detection capability of the MQ-135, it cannot measure CO₂. To obtain precise measurements of CO₂ concentration using the MQ-135, one needs to perform a manual calibration of the sensor with clean air each time the device is powered. Such a calibration process is highly inconvenient in a fully autonomous farming system that operates non-stop without requiring any startup action by humans. The MQ-135 is solely used in this project for air quality monitoring purposes because of its broad detection capability of many hazardous gases.

BH1750 -Light Intensity:

Each tray's light intensity is measured using the BH1750 digital ambient light sensor[22]. This sensor communicates via I2C and captures light intensity in lux. The sensor is precise enough that it does not require an ADC as an additional component to ensure the analog signal is digitized nor any in the field calibration. The light intensity in the sensor's range of 1-65,535 lux can go from low ambient light to high intensity in the LED growth lighting.

Water Level Sensor:

Irrigation management is informed by the water level sensor that is used to measure the liquid level at each of the trays which monitors the quantity of water for irrigation. The output signal that it produces is an analog signal which is detected by the ESP32's ADC. This data will be included in the packet of information sent from the sensors.



2.5.2 Communication Protocols for Embedded Systems

The choice of communication protocols for the inter-device links impacts the reliability, latency, and complexity of the system. The performance of several protocols was studied.

UART (Universal Asynchronous Receiver-Transmitter)

The Universal Asynchronous Receiver-Transmitter (UART) uses two signal lines to achieve simple serial communication. UART does not need a shared clock signal between devices, so it is easier to implement than synchronous methods. Both the ESP32 [16] and the Raspberry Pi 5 [17] support UART communication through the hardware UART interfaces. In addition, UART is reliable through the cable distances on a vertical farming structure. In this case, each Raspberry Pi is connected to one ESP32 using UART, and the two ESP32 nodes do not relay data between one another.

I2C (Inter-Integrated Circuit)

Inter-Integrated Circuit (I2C) is a synchronous two-way communication protocol that supports the addition of multiple devices to a single, wire-connected communication bus and the addressing of several devices on the bus. This project uses this bus feature to interface the light sensor of a BH1750 I2C light sensor and other I2C peripherals on shared wires.

RS-485

RS-485 is a standard for differential serial communication that can be used in multi-drop buses, meaning several devices can connect to the same communication line[21]. The way in which RS-485 is different to UART is by using a pair of differing signal lines, typically known as ‘A’ and ‘B’ to send data. This leads to greater noise immunity and better performance over longer cables up to a length of 1200 meters. In this project, we will utilize an RS-485 serial communication port on the Raspberry Pi 5 to communicate with the NPK soil sensor.

Most NPK sensors utilize Modbus RTU communication via a RS-485 port[20]. As the Raspberry Pi 5 lacks an in-built serial port that is RS-485 compliant, we shall require an RS-485 to USB converter or an RS-485 HAT that will convert RS-485 communications into an RS-232 type that is commonly supported by computers via their USB port. The RS-485 protocol allows an NPK sensor to communicate the amount of nutrients such as Nitrogen, Phosphorous and Potassium contained in the soil, to the Raspberry Pi 5 which then forwards the information onto our chosen database, firebase realtime database. **Wi-Fi**

The Raspberry Pi 5 uses Wi-Fi technology to connect to the Firebase Realtime Database platform.

Table 2.2: Protocols and their role

Protocol	Type	Wires Required	Role in This Project
UART	Asynchronous Serial	2 per link	ESP32-to-Pi 5 data links (independent per tray)
I2C	Synchronous Serial	2 shared bus	BH1750 light sensor on each ESP32
RS-485	Differential Serial	2 shared bus	NPK sensor to Raspberry Pi

Wi-Fi	Wireless	0	Raspberry Pi 5 to Firebase
-------	----------	---	----------------------------

2.6 Machine Learning for Tabular Data

ML has developed into a necessary technique in intelligent systems used in contemporary smart agriculture to evaluate various kinds of environmental and soil data from numerous sensors. Large scale datasets comprising structured information including temperatures, humidity, soil moistures, PH, concentration of nutrients and so forth are generated and fed into intelligent systems every moment in indoor vertical farms for producing recommendations about crop production and environmental regulation decisions. Conventional statistical as well as machine learning methods have already been extensively applied to all aspects of agricultural forecasting, yet they gradually struggle with non-linear relationships among agricultural factors and increasing level of complexity, thereby motivating the application of state-of-art deep learning. Several innovative architectures developed specifically for modeling table-shaped data have boosted the prediction performance in terms of both accuracy and interpretability in agricultural practice.

2.6.1 Classical Approaches

Initial machine learning models that aided in crop selections and environmental control in agriculture focused primarily on classical statistical and ensemble methods. These methods employed linear regression, decision trees, and support vector machines for soil and climate datasets and forecasted the optimal crop selections and favorable growing conditions. While the models were satisfactory, the complex and non-linear interactions amongst the various factors and conditions were beyond the models capabilities. The use of gradient boosting models, specifically, XGBoost and LightGBM, became prevalent on an agricultural dataset allowing the models to perform implicit feature selection at every node and naturally dealt with the inequalities that existed amongst the various factors[8],[9] These models continue to be the benchmark for deep learning models on tabular datasets.

2.6.2 Deep Learning on Tabular Data

Using deep learning on tabular data hasn't been as easy as on data like texts or images. A normal multilayer perceptron would take in each feature in an input equally, and in all layers, it would take them equally as important. There hasn't been a single neural network challenge that can clearly win against all others on tabular data, as in the literature from the early 2010s, the winner was always a boosting algorithm. Many attempts have been made to gap this challenge on tabular data, like a tab transformer as well as feature tokenization networks, but the most substantial, practical, relevant tool was TabNet.

2.6.3. TabNet

Introduced by Arik and Pfister, TabNet is a deep learning architecture specifically designed to work with tabular data[1]. TabNet's noticeability comes from a sequential attention mechanism that selects a subset of features and attends to them for each layer in the model. Because of this, all features are treated differently, and attention is given to only a svelte number of pertinent features at a time.

Consequently, this makes the model interpretable because the attention masks describe the model's thought process for each prediction. Especially in TabNet's use case concerning an agricultural control system where the operator needs to trust the AI's decisions, this feature holds prominence. Across a variety of classification and regression tasks, TabNet's authors conducted several comparative studies of TabNet's performance and found that TabNet's performance was competitive with XGBoost and LightGBM for most tabular benchmarks[1],[8],[9].

Additionally, for an example of a vertical farming system where live sensor readings are the inputs and the outputs are the environmental suggestions for crops, TabNet is the best instance of a deep learning model that is available. For training purposes, the CROP dataset contains 2,200 samples, capturing seven instances of soil and climate variables: nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall. Each sample is also labeled with a crop recommendation among 22 possible crops. The variables that these samples provide correlate with the measured variables of the NPK 7-in-1 sensor. The NPK 7-in-1 sensor also measures N, P, and K and takes a soil moisture measurement, while the DHT22 measures temperature and humidity. As such, live sensor inputs will allow little to no sensor feature modification.

2.7 Computer Vision for Plant Health Diagnosis

Computer Vision for Smart Farming A vision has long been held about the automation of crops using technology. Computer vision has led the way to many smart-farming applications including automated monitoring and diagnosing plant diseases and nutrient deficiencies by the usage of images[5],[6]. Especially in Vertical Farming, it's required to detect plants' deficiencies and diseases at the very early stage of symptoms, such that the quality and yield could be increased. Traditional human ways of assessing these are typically tedious, laborious, and also rely on the expertise of the individuals, hence this project focuses on employing computer vision using an AI-based approach to identify crop disease by taking the photos and training a model to identify visual indicators such as textures, discolorations, and other patterns and textures present on plants leaves.

Recently there have been works using self-supervised and unsupervised clustering which decreases the dependency on the use of labeled datasets which may require an expert. This section deals with all the aspects including datasets and deep learning methods and clustering approaches to automatically monitor plant health and the intelligent monitoring systems.

2.7.1 The PlantVillage Dataset

We created a work based on the PlantVillage data, which was first published by Hughes and Salathé, and includes more than 54,000 images and 26 different crops[10]. However, due to the fact that this large collection of botanical pictures was taken in a controlled environment, there is a important domain gap between training the learning models on that data and applying them to vertical farming set-ups (which typically use proprietary violet-like grow lights to nurture the cultures). We therefore created a subset of 10,000 images that include several species like tomato, citrus and corn, to have a realistic set of pictures of crops grown in conditions specific to the agricultural context. In our approach the dataset was considered an unlabeled corpus.

Using a combination of self-supervised learning and unsupervised clustering algorithms we developed an entire labeling pipeline in house using a blend of self-supervised learning and unsupervised clustering algorithms which allows us to classify images to label vertical farm cultures.

2.7.3 Self-Supervised Learning and SimCLR

Self-supervised learning does not rely on humans to mark image data, but instead develops the ability to draw useful visual representations from raw image data. SimCLR, introduced by Chen et al[2], from Google Brain, is among the most widely cited self-supervised learning framework for visual representations. It trains a CNN encoder by creating two differently augmented views of each image in a batch, and is pushed to create the same embedding for both views for the same image, while the embedding for different images are encouraged to be farther apart. The NT-Xent (Normalized Temperature-scaled Cross Entropy) is used as the training objective[2].

The augmentation also defines the irrelevant variations the encoder is learning to ignore. For this work, each original image was randomized in a vertical or horizontal flip, a random crop, jittering of the color, and a random conversion to gray scale. Each of these images of the same leaf captures the same information, but the irrelevant variation of the lighting or camera angles is different. Training the encoder to do this builds the ability to ignore the irrelevant imaging changes and instead to learn and capture the more meaningful visual data, like leaf texture, leaf pigmentation, edge conditions of the leaf, necrotic lesions of the leaf, among more.

Typically, once this process is complete, the projection head is removed, and the encoder generates a 512-dimensional feature vector for all of the leaf images. SimCLR can be trained using raw pixel data, but the input data is the output of the encoder. For the next self supervision stage, these semantically rich, high-dimensional, and array data leaf images are clustered more naturally by plant health.

2.7.4 K-Means Clustering for Pseudo-Labeling

K-Means clustering organizes data based on similarity in feature space. For our K-Means model with $K=7$, clustering was performed on the 10,000 images from the PlantVillage database, using the 512dimensional SimCLR feature vectors. An initial assessment of the mathematical robustness of the

resulting clusters showed two promising results from intrinsic metrics calculations. A CalinskiHarabasz Index of 1065.54 indicated that the clusters were dense and well-separated, and a Silhouette Coefficient of 0.18 showed that the clusters weren't of poor quality, with the relatively biological order of the clusters among stress stages providing further evidence of the robustness of the clusters.

After the initial assessment, the team validated the results by visually inspecting clusters. This included the use of visual assessment, comparison with standard agricultural stress charts, and AI-assisted evaluation to assign an agronomic label to each cluster. As a result, the pseudo-labeling of 10,000 images was accomplished. This was performed without a single image requiring a manual annotation. The efficiency of this approach was evaluated, and it was found that the interpretative work of each cluster representative was completed in lieu of the annotation of 10,000 images that were placed in 7 distinct clusters.

Table 2.3: K-Means Clustering for Pseudo-Labeling

Cluster ID	Assigned Label	Visual Justification
Cluster 6	Healthy	Uniform deep green pigmentation; no lesions
Cluster 4	Nitrogen Deficiency	Generalized chlorosis across the leaf blade
Cluster 1	pH Imbalance	Interveinal chlorosis (green veins, yellow background)
Cluster 0	Potassium Deficiency	Marginal necrosis (burnt leaf edges)

Cluster 2	Phosphorus Deficiency	Purple/bronze tints and dark necrotic patches
Cluster 5	Water Stress	Adaxial leaf rolling and wilting symptoms
Cluster 3	Temperature Stress	Large, irregular black/brown necrotic lesions

2.7.5 Lightweight CNN Architectures for Edge Deployment

Standard CNN architectures are not suited to applications such as this due to a large number of parameters and being designed purely to maximize accuracy on powerful GPU hardware. For example, VGG-16, with a total of 138 million parameters, takes about 8,000 milliseconds per inference on a Raspberry Pi CPU[5]. Even ResNet-50, with a total of 25.6 million parameters, takes about 2,000 milliseconds. They are not suitable either, for a monitoring loop that frames every 30 s, where the processor is not overloaded and the other concurrent tasks are not delayed[6].

Designed by Google to tackle these constraints, MobileNet uses depthwise separable convolutions. Performing a normal 3x3 convolution on a 64-channel feature map would require a total of $3 \times 3 \times 64 \times 64 = 36,864$ multiplications for every spatial location. Depthwise separable convolution divides this into a depthwise 3x3 convolution that requires $3 \times 3 \times 64 = 576$ multiplications, and a pointwise 1x1 convolution of $64 \times 64 = 4,096$ multiplications, for a total of 4,672 multiplications, which is 8 times fewer, with nearly the same expressiveness.

This project uses MobileNetV3-Small, which further applies neural architecture search for optimizing the trade-off between accuracy and efficiency, and incorporates squeeze-and-excitation to adaptively recalibrate the feature maps by channel, and the hard-swish activation function that is a computationally fast approximation to the Swish activation function[3].

For ImageNet pre-training, this is especially important[11]. Training a computer vision CNN from scratch on a relatively small, benchmark dataset, such as the one of plant stress, would result in significant overfitting to such a dataset. The pre-trained backbone from ImageNet is already trained to recognize edges, textures differentiating color gradients, and structural, differentiating patterns in a total of 1.28 million ImageNet photographs. These are definable features in a dataset that make the pre-trained backbone transferable and as a result, are effective for the problem that is distinct, i.e., fine-tuning on the relatively small leaf health dataset.

Table 2.4: CNN architecture comparison for Raspberry Pi edge deployment

Model	Parameters	Pi CPU Inference	ImageNet Acc.	Suitable for Pi
VGG-16	138 M	~8,000 ms	71.6%	No
ResNet-50	25.6 M	~2,000 ms	76.1%	No
ResNet-18	11.7 M	~600 ms	69.8%	No
EfficientNet-B0	5.3 M	~350 ms	77.1%	Borderline
MobileNetV2	3.4 M	~130 ms	72.0%	Yes

MobileNetV3-Small	2.5 M	~100 ms	67.4%	Yes (Selected)
--------------------------	--------------	----------------	--------------	-----------------------

2.8 Hardware Analysis

Hardware analysis is an important step in designing and developing a smart vertical farming system that guarantees seamless performance and efficiency across the range of electronic parts including sensors, controllers, communication modules, etc. Efficiently managing smart vertical farms relies heavily on the accurate hardware setup with the ability to monitor the environment conditions and to control the farming process on automated mode. This project addresses various hardware used including controllers, sensors, communication interfaces, power module and processing elements. These hardware are examined considering their function, level of precision, adaptability and costeffectiveness along with their efficiency in monitoring environment. This Hardware analysis aims to discuss the functioning and interdependence of these various hardware elements in gathering live information, processing data and conveying to the cloud for observation and action.

2.8.1 Which Microcontroller to Choose?

Multiple microcontroller platforms were analyzed to take on the sensor and actuator role. The Arduino Mega was discounted as it lacks an integrated Wi-Fi module and its single core processor cannot handle both reading and communicating sensors, which will produce serial communication conflicts. The STM32F103 has great performance capabilities, but has a more complex toolchain, a less extensive community library, and has limited local stock within Pakistan. The Raspberry Pi Zero was determined to be overqualified and overpriced for simply reading a sensor, controlling a relay, and managing basic UART communication.

It was determined that the ESP32 was the preferred option for each tray switch. Its dual core Xtensa LX6 processor that can clock up to 240 MHz allows one of the cores to be used to read sensors and the other to handle UART communication. The onboard Wi-Fi and Bluetooth were designed to be helpful during the developmental process where the Raspberry Pi 5 was not released, which allowed the ESP32 to communicate to Firebase for test purposes. The price and availability of the ESP32 was

also beneficial, as it was a requirement of the project to remain within the limited budget and hardware accessibility of students.

Out of the family of ESP32s, a comparison of available models was conducted. The original ESP32 also won out, as it has 520KB of the largest SRAM, is the best priced of all of the options, as well as being the most locally stocked.

In this project each tray has one ESP32 that's just, for that tray. The two ESP32 units work on their own. Do not need each other. Each ESP32 sends its data to the Raspberry Pi 5 using its own UART connection. This means the ESP32 units do not rely on each other to send data to the Raspberry Pi 5.

Table 2.5:Microcontroller comparison for tray-level sensing role

Feature	Arduino Mega	STM32F103	ESP32
Processor	Single-core, 16 MHz	Single-core, 72 MHz	Dual-core, 240 MHz
RAM	8 KB	20 KB	520 KB
Feature	Arduino Mega	STM32F103	ESP32
Built-in Wi-Fi	No	No	Yes
ADC Resolution	10-bit	12-bit	12-bit

UART Support	Yes	Yes	Yes
I2C Support	Yes	Yes	Yes
Local Availability (PK)	High	Medium	High
Cost (approx.)	~\$10	~\$5	~\$5–7

2.8.2 Central Processing Unit: Raspberry Pi 5

The core processing unit of the system is the Raspberry Pi 5. It has a Cortex ARM-A76 Quad Core with a 2.4 GHz processing speed and a RAM of 8 GB and LPDDR4. Because of these specifications, the Raspberry Pi 5 has higher system processing power than its predecessors. For this reason, it is well-suited for running the two inference models, that is, TabNet and MobileNetV3, in addition to running two serial UART processes and communicating with Firebase. The Raspberry Pi 5 is well-suited for this purpose, as the ESP32 tray in each front includes a UART channel, Wi-Fi integrates for Firebase connection, and CSI for a camera interface.

2.8.3 Cloud Platform: Firebase

A cloud service provider with a synchronization model that allows for a push and pull of real-time data is Firebase. Therefore, Firebase was chosen for cloud connectivity. Unlike the conventional REST models that depend on the user to repeatedly, and sometimes randomly, initiate an API call to the server for data, Firebase retains a WebSocket for its data subscribers, and when it gets new data, it is able to push that to its subscribers. The sensor data is sent to the Raspberry Pi 5, and the Firebase console and monitoring dashboard will be updated in real time.

Table 2.6:Cloud platform comparison

Platform	Real-Time Push	Setup Complexity	Cost (Small Scale)	Python SDK
Firestore Realtime DB	Yes	Low	Free tier	Yes (pyrebase4)
AWS IoT Core	Partial (MQTT)	High	Usage-based	Yes (boto3)
Azure IoT Hub	Partial (MQTT)	High	Usage-based	Yes
Custom MQTT Broker	Yes	Medium	Server cost	Yes

2.9 Development Environment

2.9.1 Arduino IDE

The primary platform used in the construction of the embedded firmware of this project was the Arduino IDE because of its ability to upload code efficiently and along with real time serial monitor. From the IDE, sensors and UART data payloads could be tested, and monitored which was highly useful while developing the necessary code for the task. The Ada emulator with large storage space, large library storage space, and a simple interface was the most optimal for this project.

2.9.2 Python on Raspberry Pi 5

Programming on the Raspberry Pi 5 was done in Python 3.11. PyTorch was used for loading and running both machine learning models, OpenCV for camera frame capture and image preprocessing, the Firebase Python client library for Firebase communication, the serial communication interface library for UART communication with both ESP32 nodes, and the standard Python threading and queue modules for task scheduling and management. Python was used because a large portion of machine learning and modeling was highly supported with an easily integrated framework to handle all jobs that needed to be done.

2.9.3 Machine Learning Training Environment

Before deployment to the Raspberry Pi 5, both Machine Learning models were trained offline on models. The SimCLR framework was utilized with MobileNetV3 fine-tuning and the TabNet model was trained using the pytorch-tabnet library. Model weights were transferred to the Raspberry Pi 5 and were set to be loaded to the memory for Edge Inference when the app was started.

Chapter-3

Methodology

3.1 Hardware Architecture

To build a truly intelligent vertical farm as opposed to an automated one, we need hardware that is capable of sensing environmental and soil conditions as to be at every level of the farm tray, continuously relaying that information autonomously and securely to a main processor, and using that information to control the operation of the hardware autonomously and respond to the AI's control instructions. This is done the system described, the AI-Driven Vertical Farming system, by an advanced parallel hardware architecture where each of the farm's 2 trays acts as a fully independent sensing and actuation unit. Each of the trays possesses its own ESP32 microcontroller, which takes reading from its full sensor, and communicates directly with the Raspberry Pi 5 over its own UART link. A camera module, acting as a sensor to the Raspberry Pi 5, provides a vital visual monitoring channel that is lost with solely relying on the sensor data.

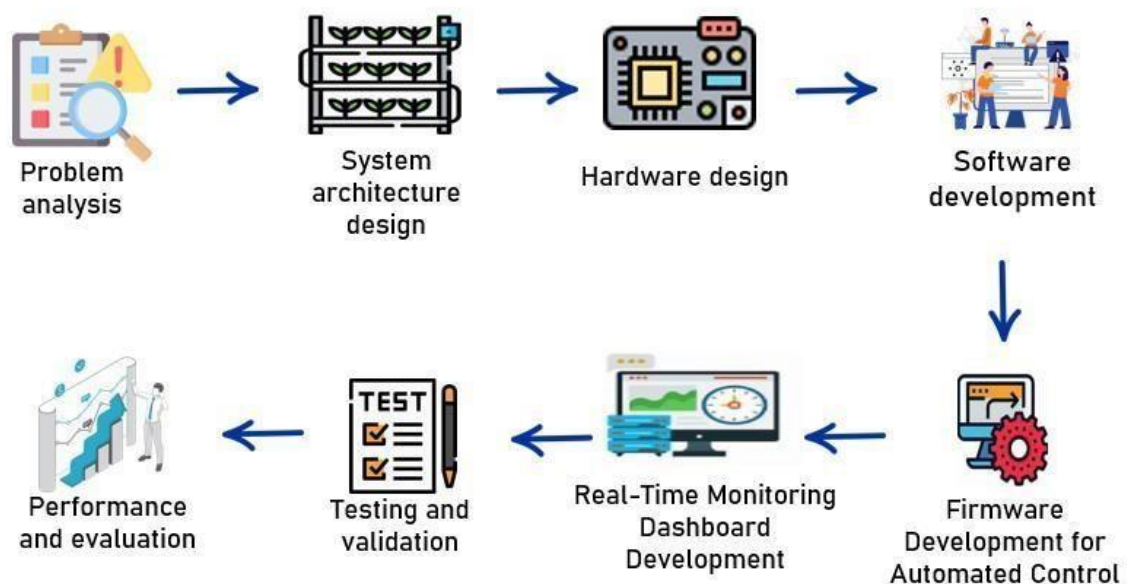


Figure 3.1: Methodology diagram

3.1.1 Hardware Components

The AI-Driven Vertical Farming system consists of the following main hardware components.

■ ESP32 Microcontroller

The ESP32 is the primary component used for each tray and is used once per tray. Each dualcore Xtensa LX6 processor can run on its own core for reading tray sensors to fulfill timing and scheduling requirements. The sensor reading core reads the sensors, and once done, sends the reading to the JSON processing core and then readies the relay control command for simultaneous UART communication to both the sensors and relays. It can read, write, and reply to control commands sent by the Pi to control the tray components relays (fan and lights).

The ESP32 units deployed at each tray are independent. Each sends sensor data to the Pi independently. The design is fault tolerant; if one tray ESP32 disconnects, fails, or is reset, the sensor data from the other tray is unaffected. The ESP32s supported direct communication to the Firebase server used for the Pi for early data verification and tests.

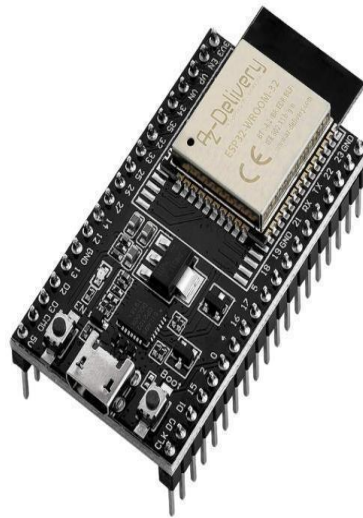


Figure 0-1.2: ESP32 MICROCONTROLLER

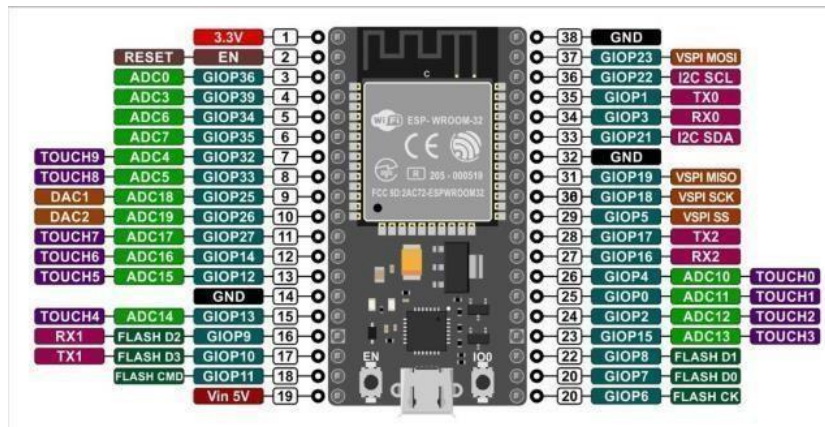


Figure 3.3:ESP32 Pinout Diagram

NPK 7-in-1 Soil Sensor

The NPK 7-in-1 soil sensor simplifies the process of measuring all parameters for individual soil trays and eliminates the necessity of building multiple sensors in the arm. The 7-in-1 sensor provides measurements of the N, P, and K content of the soil in mg/kg, soil pH, soil moisture and volumetric water content, soil temperature, and soil EC within the scope of one insertion. The ESP32 communicates indirectly with the NPK sensor, while the Raspberry Pi 5 gets the data from the NPK sensor via RS-485, implemented using a USB serial converter. One integrated sensor for the seven soil components decreases the complexity of tray wiring, and all measurements gathered from the same probe for the sake of uniformity eliminate the necessity of individual probe calibration for pH, moisture content, and nutrient levels.



Figure 3.4:NPK 7-in -1 SOIL SENSOR

DHT22 Temperature and Humidity Sensor

Each tray has a DHT22 sensor to record the temperature and relative humidity. For tray level. DHT22 has a working tolerance of approximately $\pm 0.5^{\circ}\text{C}$ for temperature and $\pm 2\text{--}5\%$ relative humidity, due to the fact that it communicates through a one-wire digital system. These parameters assist in not only the control of fans and optimum orientation of the shutters, but also the prediction of the TabNet environment.

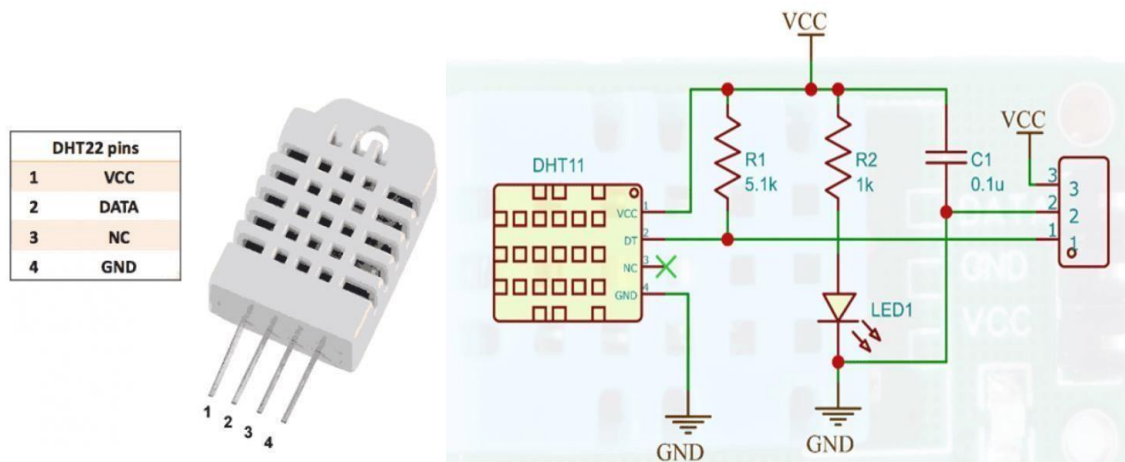


Figure 3.5:DHT22 SENSOR

MQ-135 Air Quality Sensor

The MQ-135 gas sensor categorizes a variety of gases present in a closed environment. It is sensitive to ammonia gas, smokes, benzene, alcohol, and other gases. This sensor is used in this project to monitor the air quality in the tray enclosure by detecting the presence of harmful gases and the degradation of air quality which can be harmful to the plants. This sensor is not used here to measure the concentration of CO₂ gas. The MQ-135 gas sensor does respond to CO₂ gas, but since this sensor is broad and sensitive, the use of this sensor for CO₂ gas measurement will mean that the baseline will have to be set by the user every time the power is turned on. This is lots of impractical work especially for an autonomous system. The MQ-135's output will be in the form of an analog through the ADC of the ESP32, which is a 12-bit ADC and will be used to monitor and display the air quality index.



Figure 3.6: MQ-135 AIR QUALITY

BH1750 Light Intensity Sensor

The light intensity of each tray is monitored using the BH1750 digital ambient light sensor. This sensor communicates through the I2C bus and measures light intensity and outputs its lux reading in a digital form. This sensor has a measurement range of 1 to 65,535 lux which is able to measure all ranges from low ambient light to fully lit lights. The light intensity of each tray is measured to confirm that the grow lights that have been set up provide sufficient photon flux to the crops that are growing on that level.

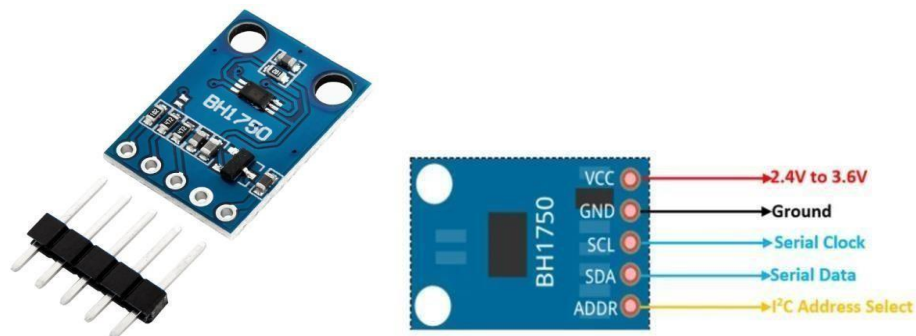


Figure 3.7: BH1750 LIGHT INTENSITY SENSOR

Water Level Sensor

We placed one water level sensor in each of the trays in order to give us an idea of the space left for water to be added. ESP32's ADC samples the output from the water level sensor, and the data is integrated in the sensor data packet being sent to Raspberry Pi 5. Constructing irrigation control decision making would be possible with the data, but the water level sensor does not provide a warning for the reservoir, as it is not concerned with water being stored only with the tray.

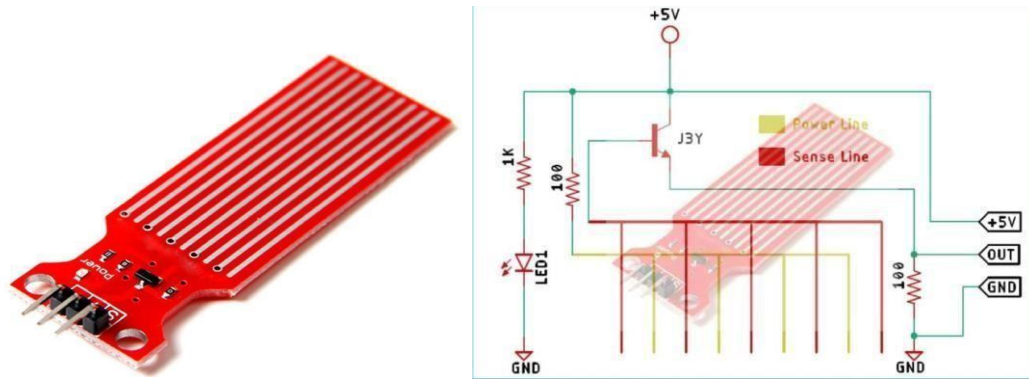


Figure 3.8: WATER LEVEL SENSOR

Actuators

The actuators on the trays (a fan for temperature and ventilation, an LED grow light strip for light intensity, and a water pump for irrigation, a heater for dealing the low temperature conditions) are each connected to the ESP32 via channels on a relay module. The relay module enables the ESP32's 3.3V logic level signals to control the higher voltage circuits that drive the devices, without there being any electrical connection between the microcontroller and the loads. Each of the actuators is connected to an ESP32 GPIO. When a command payload for the actuators is received by the ESP32 from the Raspberry Pi 5 via UART, the ESP32 reads the corresponding fields of the command and toggles the appropriate relay channels to physically change the conditions of the growing environment setup on its tray.



Figure 3.9: ACTUATORS; Heater, Fan, Pump, LED

Raspberry Pi 5

The control center of the whole setup is the Raspberry Pi 5. Its quad-core ARM Cortex-A76 processor with a 2.4 GHz clock speed and a generous 8 GB of LPDDR4X RAM allows it to perform several

functions simultaneously, such as receiving independent UART data from both ESP32 nodes, carrying out the TabNet inference model to make predictions, running diagnostics with the MobileNetV3 model, and performing read and write functions to and from FBD. There is also the additional task of sending actuator command payloads to the ESP32s on the respective UART lines. The Raspberry Pi 5 controls its connection to FBD over Wi-Fi. The Pi also serves as a host for a camera module and provides the necessary power to it. The camera is actually a USB camera, which connects directly to the USB port.

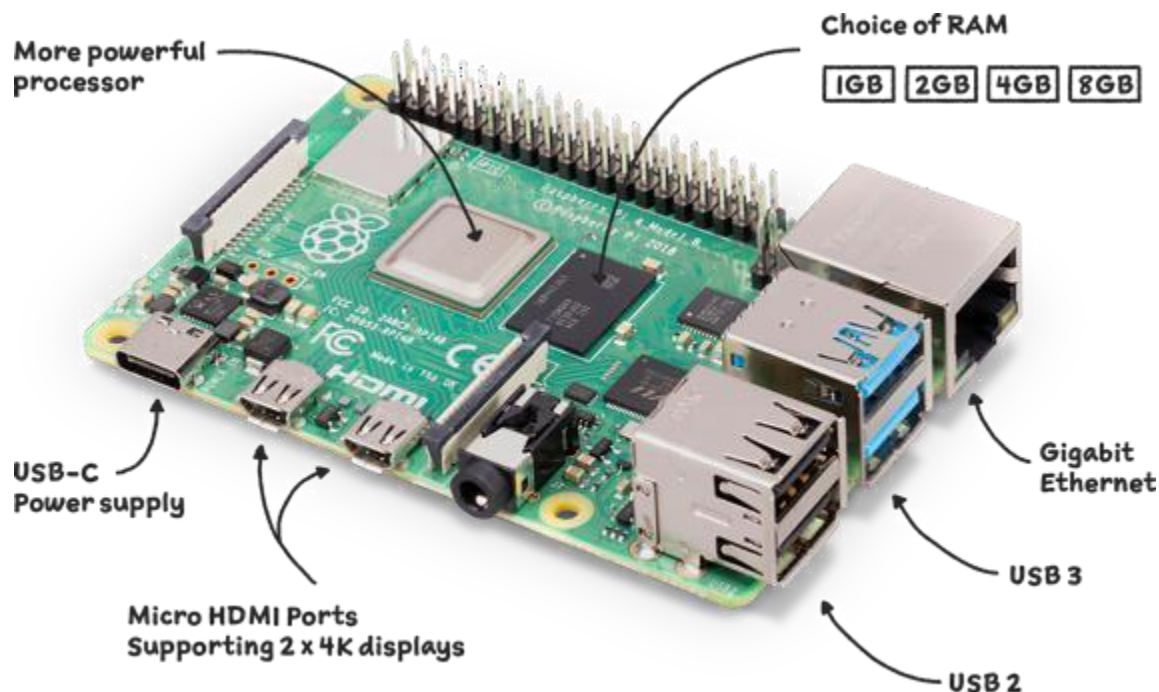


Figure 3.10: Raspberry Pi5

Camera Module

The ability to stream a constant live feed to the area being monitored occurs thanks to a USB camera directly linked to the USB port of the Raspberry Pi 5. Once every thirty (30) seconds, the camera captures a frame to be processed through the (image) processing pipeline (e.g., grow light correction, segmentation) and classified by the MobileNetV3 model to assess the health of the plants. Once processed, MobileNetV3 results are saved to FBD and shown on the Pi 5's USB camera in a camera view to allow the system operator to assess the health of the plants for the absence of sensor data.



Figure 3.11: Camera module

3.1.2 System Overview

The vertical farming system has two trays. Each is integrated with sensors, relay-controlled actuators, and an allocated ESP32 unit. The Dual ESP32s work in parallel, gathering sensor information from their respective trays, and relay it to a Raspberry Pi 5 over separate UART connections. The Raspberry Pi 5, simultaneously receiving both data streams, combines the data and sends it to Firebase to create a real-time display. The Raspberry Pi will also run the TabNet model to provide suggestions on environmental changes regarding specific crops, and will ultimately send commands to the respective ESP32s. The Raspberry Pi will also be linked to a camera module. It will be on a different interval, capturing images of the crops to determine plant health. MobileNet V3 will classify the leaf images. The determination will precede posting the results in Firebase. The Raspberry Pi connects to Firebase over Wifi.

3.1.3 Hardware Block Diagram

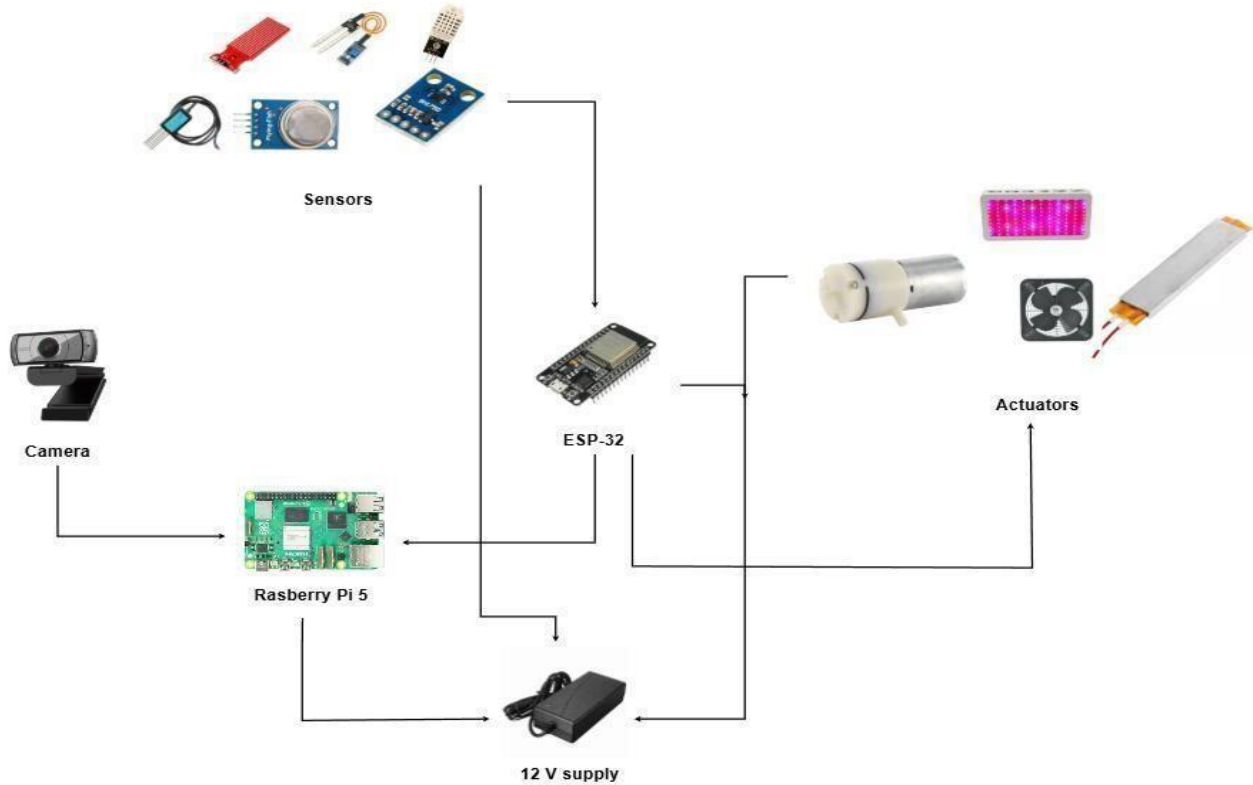


Figure 0-2.12: Hardware Block Diagram

Both trays' ESP32s connect to the NPK sensor via RS-485, the BH1750 via I2C, the DHT22 via digital GPIOs, the MQ-135 and the water level sensor via the analog pins, and the relay via the GPIOs. Each ESP32 UART TX pin connects to independent UART RX pins on **the Raspberry Pi5**. The UART TX of the Raspberry Pi connects to the RX pin of the ESP32, creating a cycle of commands.

3.1.4 Flowchart

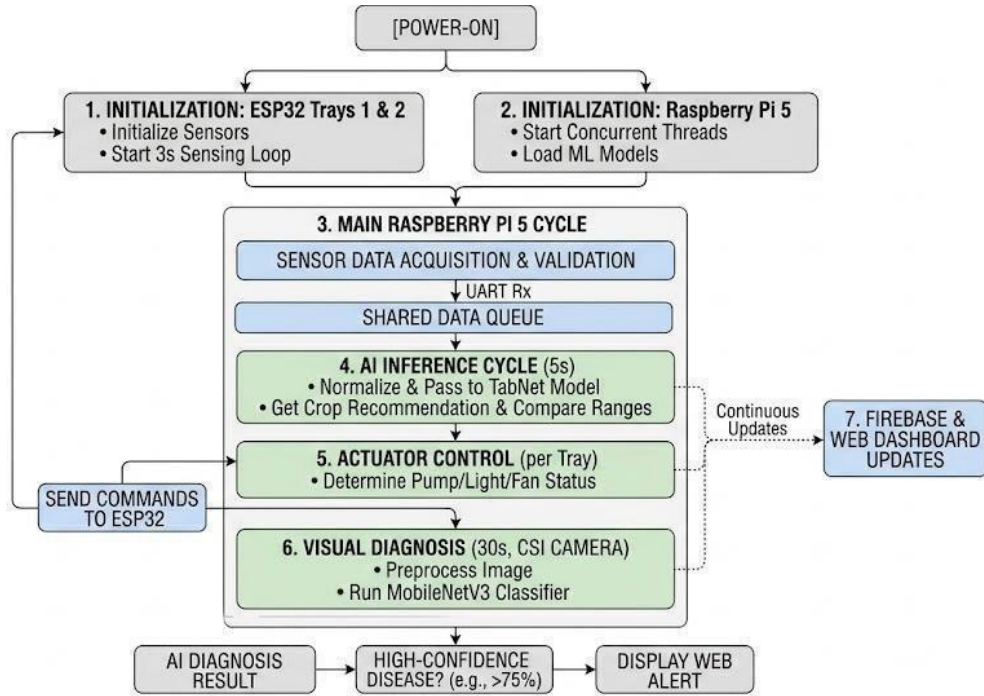


Figure 3.13: Flowchart

3.2 Software Architecture

The AI-Driven Vertical Farming system relies on a vertically-integrated, four-layer, cooperating software architecture: the embedded sensing layer, edge intelligence layer, cloud storage layer and the user-facing monitoring layer. Each layer has clearly bounded responsibility and interact with neighboring layers via well-defined interface thereby assuring that a failure and/or an update of one layer is contained. This division of concern is by design to let each layer evolves independently whilst the overall system continue to run normally.

The embedded layer consists of two ESP32s microcontroller based units (1x/tray) which acquire timely environmental readings from the on tray sensing infrastructure, and communicate with the on Raspberry Pi 5 host machine, by way of separate UART connections. In the edge layer a pair of AI inference pipeline processes on Raspberry Pi 5 serve two distinct objectives and run in parallel as distinct Python services: `pi_tabnet.py` a TabNet based forecasting pipeline making recommendations regarding plant parameters optimized according to the operators choice of crop, and `pi_cnn.py` a CNN based visual inspection pipeline performing continual surveillance of a video feed identifying

symptoms of visual plant stress; each service accesses sensory inputs and writes predictions on dedicated path on Firebase. The cloud layer is the cloud database system based on Firebase Realtime Database acting as the global shared memory of the system: all components read and write from and to Firebase to communicate one with the other; i.e. no layer directly communicates with any other layer (except its direct neighbors). Finally the presentational layer is a React web UI, running on Vercel and subscribing to all desired Firebase path(s) by use of the Firebase JS SDK, rendering the readings, inference outputs and control interfaces for the operator.

Our architectural pattern also ensures graceful system degradation in event of partial failure. If the camera feed become invalid, our CNN automatically reverts to a sensor-gate only diagnosis. In event of unavailability of Firebase, both raspberry pi's AI services remain operational on their respective on-board caches and. Should either one of our AI services accidentally crash, their respective service are auto-restarted by way of system-managed process in under 2 sec without requiring human intervention.

Table 3.1:Four-layer architecture summary

Layer	Technology	Responsibility	Communication
Embedded	ESP32 + Arduino Firmware	Sensor acquisition, JSON upload	Wi-Fi to Firebase
Edge AI	Raspberry Pi 5, Python	TabNet + CNN inference	Firebase read / write
Cloud	Firebase RTDB	Data synchronization bus	WebSocket push

Presentation	React / Vercel	Live dashboard, crop selection	Firebase JS SDK
--------------	----------------	--------------------------------	-----------------

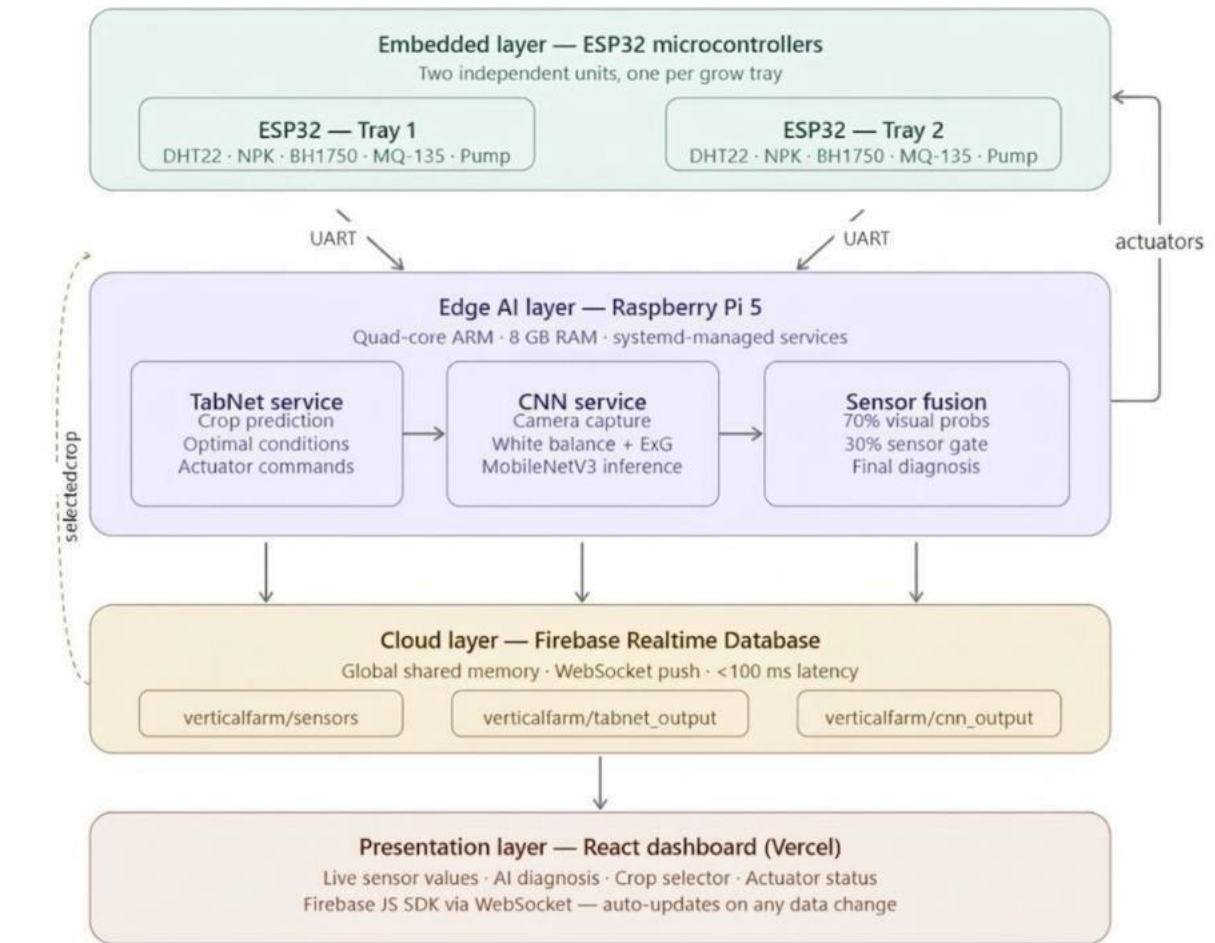


Figure 3.14: Software Flowchart

3.2.1 ESP32 Firmware

The sensing tier of the proposed system is based on two ESP32 development boards programmed in C++ under the Arduino framework; The choice of ESP32 development boards was made based on the ease of development afforded by the platform, its extensive sensor library ecosystem, and builtin debugging via the Serial Monitor that proved invaluable during repeated testing of the sensor data streams and communication payload. Each ESP32 board was tasked to monitor a single growing tray thus creating a distributed architecture where sensor data is collected locally and the command is

issued locally. The Arduino main execution loop polling all attached sensors at a three-second interval provides near-real-time sensing data from three different sensing points, (e.g. DHT22 for temperature, BH1750 light sensor for ambient light intensity through an i²c connection to the ESP32's pins and a NPK 7-in-1 soil sensor through an rs485 transceiver operating using the Modbus RTU master framework), all additionally along with the NPKs analog sensors by using the ESP32's built in analogue-to-digital (ADC) convertor; The values from the analogue sensors i.e. Water Level Sensor, and the MQ-135 Air Quality Sensor were then converted, using appropriate calibration and firmware level compensation, into analogue-digital values. In addition, all sensors readings were verified for physical and/or operational limitations to detect faulty or unreliable measurements. If a value is deemed to lie outside its plausible range it is marked as null; This was deemed preferable over trying to average unreliable measurements for fear of propagating erroneous information to the next stage of data processing i.e. the Ai inference component that is most sensitive to incorrect data, especially for small models. All validated sensor readings are then encapsulated into a JSON formatted data packet with newline characters at the beginning and the end indicating the start of the payload and the end respectively. The encoded payload was then transmitted to the Raspberry Pi 5 on a single UART port assigned to each specific ESP32 (UART0 & UART1). Furthermore, every ESP32 was constantly checking its own UART receive buffer for an incoming payload from the Raspberry Pi; if found, the payload was decoded, and the appropriate GPIO pins were toggled on or off to control relays on each respective grow tray activating physical outputs such as the cooling fans, led grow lights, and the irrigation water pump. A major architectural constraint/choice of this system was that the ESP32's had no form of communication amongst themselves but instead communicated directly and exclusively to a Pi on a dedicated port (i.e. a one to one connection for each ESP32); a more fault resilient and a distributed system can be achieved this way, as if one ESP32 crashes or is reset, the system as a whole is not at risk since the other ESP32 continues its task without being disturbed.

3.2.2 Central Coordination

The system's computational core is the Raspberry Pi 5. It manages two standalone services, `pi_tabnet.py`, and `pi_cnn.py`, both of which are handled by the `systemd` service manager. `Systemd`

ensures each service starts during the boot process, automatically restarts the services after 2 seconds of a crash. While they both utilize Firebase as a data bus, they don't communicate or have any assumptions about the state of the other service. This enables the services to be restarted, updated, or debugged without impacting the other service.

3.2.3 UART Communication and Actuator Feedback Loop

The Raspberry Pi 5 communicates with each of the ESP32 actuator controllers for individual trays light-dimmer, nutrient-dosers and fans via dedicated UART serial channels. The TabNet service sends computed set-points for temperature, humidity, pH and NPK levels to Firebase at address suggested conditions. A dedicated controller process on each tray reads this from Firebase, compares it to realtime readings, and sends commands to the ESP32 relay drivers over the appropriate serial link. It implements a PI controller for each controlled variable with proportional and integral gain tunable constants so that set points are reached smoothly. For example, if current temperature for tray 1 is 29 C, but the crop is expecting a temperature of 25 C, the controller calculates a fan-speed command proportional to this difference in addition to an accumulated offset term which helps reduce steadystate error. Control loops run at 1Hz, which is fast enough to maintain stability for the slowly changing conditions inside an enclosed growing chamber.

3.2.4 TabNet Environmental Prediction Pipeline

This AI pipeline predicts the optimal set of growing conditions for a selected crop. It uses the TabNet architecture because of the nature of tabular data as opposed to simpler machine-learning models such as Random Forest and XGBoost. Firstly, TabNet uses a sequential attention process that identifies only the most important input features for each prediction, helping with interpretability by visualizing which sensor data matters most. Secondly, it's an attention based model uses end-to-end deep learning and eliminates manual feature engineering by finding complex non-linear relationships within the raw sensor data itself, no assumptions needed. Thirdly, it's PyTorch based Architecture is compatible with the same training pipeline infrastructure used for the CNN that was chosen.

- **Feature Normalization and Preprocessing**

Each row in the Crop Recommendation dataset details the condition in which a crop is grown with the following features: nitrogen (N), phosphorus (P), potassium (K), temperature, humidity, pH, and rainfall. The dataset includes the crop grown as the label. During the pre-processing of the dataset, missing numerical values are replaced with the mean of that column, while missing categorical data are replaced with the modal value. The crop labels were then integer encoded. The integer encoder is saved so the integer crop label predictions can be converted back to the crop label. After the integer encoder is saved, before the training of the model, the features in the dataset have a Quantile Transformation which transforms all the data to a Normal Distribution. The Quantile Transformation with a normal distribution rely less on the range of the data. The outliers are less emphasized. The training dataset is used to train the quantile transformer to prevent data leakage, and then the training and testing dataset split is stratified to create eight parts of the training and testing dataset. The trained model is then used for new predictions while the trained quantile transformer and label encoder are also used for returning readable predictions.

- **TabNet Configuration and Training Methodology**

The TabNet classifier hyperparameters were determined through a combination of published recommendations from the original paper and grid search over a validation subset.

Table 3.2: TabNet hyperparameter configuration

Hyperparameter	Value	Rationale
n_d (decision step width)	32	Balanced capacity vs. Pi inference latency
n_a (attention embedding width)	32	Equal to n_d per original paper guidance

n_steps (sequential steps)	3	Adequate for 7-feature tabular input
gamma (sparsity coefficient)	1.3	Encourages sparse feature selection per step
max_epochs	30	Sufficient for convergence on this dataset size
Hyperparameter	Value	Rationale
patience (early stopping)	7	Triggers consistently between epochs 20–25
batch_size	128	Fits in Pi 5 RAM; acceptable gradient estimate
device	cpu	Pi 5 has no GPU; CPU inference is adequate

The entire model is implemented with the PyTorch TabNet library, which extends the PyTorch framework and presents an accessible API, very much like scikit-learn’s `fit()` and `predict()` interfaces, while making sure that the main work can be summarized in training a deep neural network, optimizing a given prediction loss (cross-entropy), while promoting variable selection at the same time. The latter goal is achieved by introducing sparsity to attention masks of the model by adding an L1 penalty to their value during optimization. Variable selection is controlled by means of hyper-parameter (gamma) that trades-off prediction loss and a penalty for feature

importance. After training, the model is serialized using the TabNet built-in `save_model()` function into a compressed archive containing the PyTorch state dictionary and all metadata necessary for its configuration. On the Raspberry Pi 5, when the service starts, it checks for the existence of this archive if the archive is present, it will be loaded by `load_model()` in under two seconds, otherwise a fresh training cycle is initiated before entering the inference phase.

Automated Environmental Control Logic

The inference stage of the TabNet pipeline consists of two discrete parts. In the first part, the current sensor readings are preprocessed by the saved Quantile Transformer, and are passed through the trained classifier which outputs the (integer) class index of the optimum crop. This class index is converted to human-readable form by the `inverse_transform` of the Label Encoder, and saved under the key 'verticalfarm/aioutput/predictedcrop'. The second part computes suggested optimal operating conditions for the crop chosen by the operator using the dashboard. This is calculated by taking the column-wise average for all training data instances corresponding to the chosen crop, and then truncating (clipping) each resultant value into physically reasonable ranges. A data based and deterministic approach was taken, rather than optimization based on input gradients, as previously considered (during the earlier days of development). We rejected the latter approach after finding the previously evaluated optimization of input gradients to be not feasible due to its low testability (at slow speed on the Pi CPU this process was completely erratic resulting in very large variations), as well as requiring very forceful clamping to keep output values within the bounds of physical realism. Calculating the mean over the rows of the training dataset was a process which is carried out quickly and accurately by the pandas library, was entirely reproducible and is directly related to the spread of physical conditions under which individual plants were recorded during training data capture. The predicted settings will be sent to Firebase and will be read by the actuator control module to control the relays of the plant growing system.

3.2.5 Visual Plant Health Diagnosis Pipeline

The second AI pipeline takes in images captured by the camera module and using a Convolutional Neural Network it detects any symptoms of plant stress. Unlike the sensor-based TabNet pipeline

which works with raw, numerical inputs, the visual pipeline takes live camera feed as input and extracts visually detectable and indicative information such as, leaf wilting/deformation, discoloration, and the presence of necrotic spots which cannot be directly detected by any physical sensor available in the plant system. Finally, both the visual diagnosis result and the sensor-based classification from the TabNet pipeline are combined in a decision layer. The resulting fused diagnosis is saved under the key 'verticalfarm/aioutput/plantstressclassification'. Given the constraints imposed by the target environment – in the plant-growing chamber there are only purple grow lights, a photometric normalization preprocessing stage is required to correct the image color skew produced by the lights, thereby allowing color based processing without making problematic assumptions about expected plant leaf green color.

SimCLR Self-Supervised Pretraining

SimCLR, a "Simple Framework for Contrastive Learning of Visual Representations," was used to perform pre-training of the convolutional neural network backbone using 10,000 images from the PlantVillage leaf disease image repository. No explicit labeling information was leveraged in this training phase, and the CNN encoder learns a set of generalist feature extractors. SimCLR trained to group two random augmentations of each image more closely together, in an Ndimensional projected embedding space, than any two random augmentations of different images within the current batch. The model is trained on an object, by learning the similar representations with differences as images by the same subject from different Augmentations as positive samples and others from another subject from another as negative samples. A contrastive learning objective is applied, the NT-Xent loss function. Minimize the loss encourages model bringing similarity images closer while pushing difference images faraway. The temperature factor $\tau = 0$.

5 was used to define the contractiveness with a proper scale. Since images captured under LED lights are susceptible to huge color variations depending on lighting condition, viewing angle and the properties of surface of leaf, we adapt the traditional data augmentation procedure by increasing the value of hue variation in to see much bigger color variations in the process.

Additionally, random hue shift also employed, because red/blue lights vary for different LED fixtures. This strategy allows the model to focus on structure rather than color of leaves.

Upon completion of training, the learned projection head will be dropped, leaving just the main encoder for future usage.

PCA Dimensionality Reduction and K-Means Clustering

Prior to supervised fine-tuning, the abovementioned pre-trained encoder is used to segment the unlabeled dataset into logical groups (Pseudo-labels). All 10k images are fed through the trained network in evaluation mode, to create a $10k \times 576$ high-dimensional feature embedding matrix. The embedding dimension was then reduced by PCA into the 50 top principal components, retaining approx. 85% total variance, before standardization (using standard Scaler) and clustering of embeddings. Clustering was carried out using Mini-batch k-means for 20 random initializations and selecting the best outcome according to its final inertia. Because our data-set is large and its high-dimensional output requires significant memory footprint, Mini-batch KMeans was selected over K-Means to process data in chunks. Two indices were used to assess mathematically the clusters before visual analysis: Calinski-Harabasz index of 1065.54 (strong separated clusters) and Silhouette score of 0.18 (low average silhouette suggesting natural transition between adjacent classes). The leading representative members for each group were later selected and visually labelled according to existing agronomical knowledge; forming our structured labelled dataset ready for supervised learning on the model-fine tuning stage.

Multi-Head MobileNetV3-Small Architecture and Training

To adapt the MobileNetV3-Small (mNAS) backbone for object detection, the originally implemented 1,000-class ImageNet output head is removed and replaced with three task-specific heads which are implemented on top of the network's shared 576-dimensional global average pooling output.

Table 3.3:Multi-head MobileNetV3-Small output head architecture

Head	Output	Loss Weight	Purpose
Classification Head (cls_head)	7-class logits	40%	Visual-only stress classification
Regression Head (reg_head)	Severity score [0,1]	10%	Stress intensity estimation
Decision Head (decision_head)	7-class fused logits	50%	Primary fused diagnosis output

To distinguish between cases that the network cannot visually classify, we embed IoT sensor information into the decision head. This is in contrast to standard BatchNorm1d that does experience a runtime crash under such circumstances. The specific examples of yellowing leaves include scenarios with severe nitrogen deficiency versus stress from water deficiency, where the ‘NPK’ sensor can measure the nitrogen level in soil, distinguish from soil moisture levels what is happening to the plant.

Table 3.4: MobileNetV3-Small fine-tuning training configuration

Training Parameter	Value	Rationale

Optimiser	AdamW	Weight decay prevents overfitting on small labeled set
Learning rate	$1e-4$	Small to preserve pre-trained ImageNet backbone features
Weight decay	$1e-4$	Regularisation for fine-tuning
LR Scheduler	CosineAnnealingLR	Smooth decay from $1e-4$ to ~ 0 over full training
Sampler	WeightedRandomSampler	Corrects severe class imbalance in plant disease data

Gradient clipping	Max norm 1.0	Prevents random heads destabilising pretrained backbone
Stage 1	Backbone frozen, heads only, 10 epochs	Initialises head weights before full finetuning
Stage 2	All layers unfrozen, 50 epochs	Full fine-tuning at low learning rate
Best checkpoint	Highest validation accuracy on Decision Head	Saved as growbotos_model.pth

📷 Image Preprocessing Purple Light Correction

Every camera image goes through a two-stage preprocessing procedure before entering the CNN.

Stage 1 Grey-World White Balance Correction: In this procedure, our “grey-world” heuristic tries to create a more natural looking image with plants looking green and resembles the training dataset. It normalizes channel means with total image mean and corrects for under-exposed(g) or over-exposed (r,b) channels by applying corrective multiplication factors. For matching training domain under purple light, we introduce a pre-training pipeline on each image from the dataset by multiplying red channel by 1.45, green channel by 0.35 and blue channel by 1.30 before augmentations.

Stage 2 ExG and HSV Dual-Mask Segmentation: This procedure is performed to isolate the leaf from background. The Excess Green Index mask [29], highlights green pixels. A secondary, HSV based mask [30], takes over for color-separated information: primary green band (hue 30°–90°), yellowish green band for early chlorosis (hue 20°-35°) and the residual pink-magenta region, likely representing waxed areas of the plant. These masks are combined via bitwise OR to generate the image mask, which is then cleaned through opening and closing morphological operations before background is rendered black. The image is then resized to 224x224 and normalized using ImageNet standard channel statistics.

Multimodal Sensor Fusion and Decision Gating

Finally the diagnosis decision is obtained through a two part model fusion; the neural part generates a 7-class softmax probabilities from its 7-class Decision Head logits; each live sensor output data is read from its physical device and independently generates a 7-class probability vector that de-emphasizes the “Healthy” class based on breach of their specific operative threshold. Combining these outputs at a 70%-30% split provides the final probability distribution, where the 70% gives dominance to the visual diagnosis and the 30% to any conclusive sensor diagnostics. The fused probability vector is: **final_probs = 0.70 × neural_probs + 0.30 × sensor_gate** .

The argmax is selected from the final distribution. Additionally, an operator may find additional context in the human readable output from selected sensors. The raw sensor readings associated with the current prediction state (i.e. ‘LOW TEMP (38.2C)’ for high temp prediction) is appended to the prediction and uploaded to Firebase and displayed on the dashboard. Our complete inference result log includes the prediction name, confidence score, severity level intensity, the full set of class probabilities, active sensor flags, camera connect status and timestamp information.

3.2.6 Cloud Integration and Monitoring Ecosystem

All persistent data, including current sensor readings, AI pipeline outputs, operator setpoints, and system health checks are stored in Firebase Realtime Database. Alternative cloud services like AWS IoT Core or a self-hosted MQTT broker were evaluated and dismissed in favour of Firebase because: First, the code can access the Database from any of the three tiers of computation using JavaScript, Python, or C++ using uniform API calls to the SDK for each language. The free “Spark” tier of Firebase has more than enough capability to handle the amount of data required to run a two tray prototype.

Firebase Realtime Database Schema

The database is split into two root-level entities. Sensors/ holds the readings from each per-tray sensor which are each ESP32 patches to this branch atomically, the vertical farm entity holds all application state which is maintained by the Pi services and web dashboard.

Table 7: Firebase Realtime Database path schema

Firestore Path	Writer	Contents
Sensors/Tray1	ESP32 node 1	temperature, humidity, air_quality, light_lux, N, P, K, pH, moisture, soil_temp, conductivity, water_level
Firestore Path	Writer	Contents
Sensors/Tray2	ESP32 node 2	temperature, humidity, air_quality, light_lux, N, P, K, pH, moisture, soil_temp, conductivity, water_level

verticalfarm/selectedcrop	Web dashboard	Crop name string chosen by operator
verticalfarm/aioutput	pi_tabnet.py	predictedcrop, suggestedconditions, livesensors, timestamp
verticalfarm/cnnoutput	pi_cnn.py	stress_type, confidence, intensity, all_probs, sensor_flags, camera_connected, timestamp
verticalfarm/tabnetstatus	pi_tabnet.py	Service lifecycle status (starting, ready, error, stopped)

The React web dashboard that is deployed to Vercel, subscribes to all the interesting branches in the Firebase store using the Firebase javascript SDK.

3.2.7 Design Considerations

- **Modularity**

Each component of the system consists of independent (clearly defined) modules that are used to define: ESP32 firmware; TabNet service; CNN service; Firebase schema; React Dashboard. Each module can be developed, tested, updated, or replaced independently from each other since changes to any one will not affect any of the others. The value of modularity was evident during development, where changes made to the CNN training pipeline did not require any modifications to either the TabNet service or the ESP32 firmware.

- **Scalability**

A third plant tray could be added by installing a third ESP32, defining a third path in the

Firebase structure (Sensors/Tray3), and a third UART read thread to the Pi application without modification of existing modules. The Vercel-hosted React Dashboard will scale automatically with user traffic and no operational server management required

Robustness and Graceful Degradation

The system was engineered to respond gracefully to component failure. If the camera fails, diagnosis will rely on sensor data alone. If the system loses access to Firebase, Pi services may operate from local caches of recent readings. If either Pi process dies, systemd will restart it in less than 2s. Failed sensor readings on the ESP32 produce an invalid-data-marker value rather than a false measurement value and no transmission.

Chapter-4

Results and Discussions

4.1 Introduction

Here, we show the experimentally obtained results on the two key AI components developed for the AI-Driven Vertical Farming system the self-supervised clustering pipeline used to create the labelled plant health dataset and the MobileNetV3-Small CNN model trained on that dataset for visual plant stress identification. We report on the results as they were encountered throughout our development pipeline; that is, the results from the clustering evaluation process first, followed by the analysis of dataset balancing, and the evaluation of the performance metrics during CNN training and testing. Each result is evaluated for both its numerical magnitude and its impact on the capacity of the system to deliver accurate plant health diagnostics in practice.

4.2 Clustering Results

Clustering plays an important part in the automatic plant health diagnosis system in grouping visually similar images of the same plants together, without human intervention to assign labels. Employing the feature representations acquired using the SimCLR model, the clustering algorithm attempts to find patterns and similarities among the plant images according to their visual appearance and condition. This is critical to successfully producing pseudo labels that subsequently feed into the learning model. In this section, various clustering indices are utilized to assess the performance of the clustering process by quantitatively assessing the separation of clusters, the compactness of individual clusters, and the internal cohesiveness.

4.2.1 Overview

Our entire pipeline of visual diagnosis of the visual health information in our captured images rests heavily on the clustering process. Having supplied unlabelled images to the algorithm, the success of the final labels will depend primarily on the accuracy of K-Means clustering algorithm to group images based on visual similarities and underlying similarities in the SimCLR feature space. To determine the overall quality of the generated clusters before manual curation or label generation, three popular metrics of intrinsic clustering quality evaluation are computed.

4.2.2 Clustering Evaluation Matrices

```
=== CLUSTERING EVALUATION METRICS (for your report) ===  
Silhouette Score      : 0.1836 (higher = better)  
Davies-Bouldin Index  : 1.8673 (lower = better)  
Calinski-Harabasz     : 1065.54 (higher = better)
```

Figure 4.1: Clustering Evaluation Matrices

These metrics were used to gauge the model's clustering performance: The highest score in the clustering evaluation is the Calinski-Harabasz Index of 1065.54. This metric reflects the degree of separability between clusters and the internal compactness within each cluster; high values correspond to well-defined clusters. It indicates that the model learned distinguishing characteristics, grouping images of leaves that were visually and biologically similar.

The Silhouette Score of 0.1836 is an indication of moderate to positive clustering performance. This metric quantifies the similarity between data points and their assigned clusters compared to neighboring clusters (range -1 to 1). This is a relatively low but acceptable value for biological data due to continuous variations within plant health classes. For instance, leaf transitioning between healthy stage to early stage of nutrition deficiency often resembles as visually similar in terms of color to both conditions. The score simply means the natural variation within biological phenomenon, and is not indicative of a bad clustering.

The Davies-Bouldin Index score of 1.8673 suggests similar partitioning where a good separation of clusters corresponds to values lower than a certain reference, like 1.8673 itself indicating some proximity among the neighboring clusters (often observed due to indistinguishable characteristics between conditions such as nitrogen deficiency & pH imbalance resulting in yellow leaves). In our system, additional sensor data (NPK and pH) were combined with leaf images to achieve a reasonable classification accuracy. In general the average Silhouette scores in all of the evaluation showed that

clustering produced seven valid and distinct groups within the leaf data as opposed to the conventional way of labeling all data first, then perform classification to the labelled data set.

4.2.3 Distribution Of images

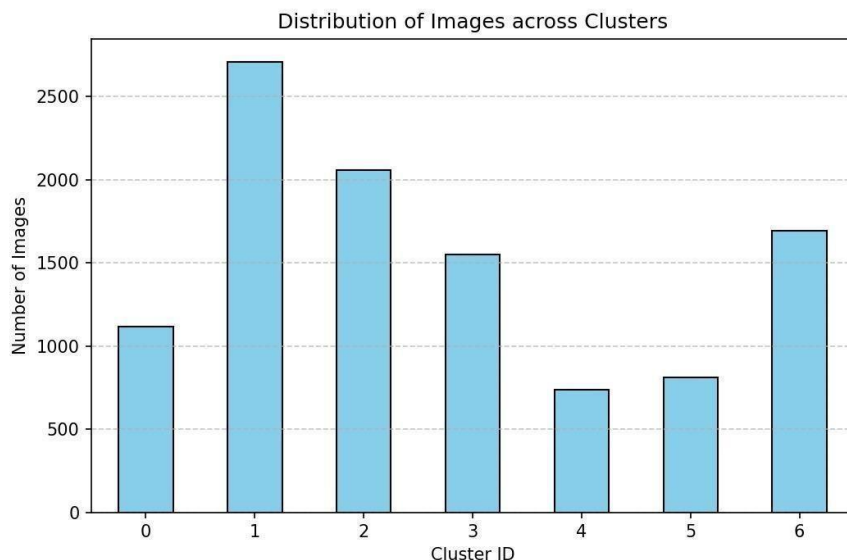


Figure 4.2: Distribution Of Images Across Clusters

The distribution of images across the seven categories was highly skewed. Cluster 1 (labelled PH imbalance) had the highest number of images (2714, representing 25.4% of the dataset) and Cluster 4 (labelled N Deficiency) had the fewest (746 images, representing 7.0%). This was not wholly unexpected as the PlantVillage dataset, from which these images were sourced, has inherent class imbalances representing the real-life frequency of each stress in the original image acquisition. The typical symptom of PH imbalance (interveinal chlorosis often with a yellow-between-green-veins signature) occurred across various plants in the dataset, leading to a large cluster, whereas the relatively homogeneous symptom of N deficiency (generalised yellowing) led to a smaller, more compact cluster.

This distribution had direct practical implications if we wanted to train a CNN on the data - training on this imbalanced set would strongly bias the model towards the majority class or classes. We addressed this imbalance with the process described in Section 4.3.

4.3 Dataset Balancing

4.3.1 Motivation for Balancing

When classifying on an imbalanced dataset, the learned decision boundary will be biased towards majority classes. This is especially problematic for plant health diagnosis; the minority classes such as Nitrogen and Phosphorus deficiencies (together representing just 14.5% of the raw clustered set) are arguably the most important to detect because there is often significant agronomic value to early diagnosis. If a model could accurately diagnose 2700 images of PH imbalance, but could only diagnose 746 images of N deficiency correctly, then the model is failing one of its core objectives.

4.3.2 Method and Result of Balancing

To balance the dataset, a balancing step was inserted after clustering and label assignment. All seven categories were clipped at 737 images. For categories larger than 737 images, the data were randomly down-sampled (without replacement) to 737. All images for classes which originally had fewer than 737 examples were retained in their entirety. The resulting dataset was balanced, with an equal number of images for each of the 7 stress classes used in the classification.

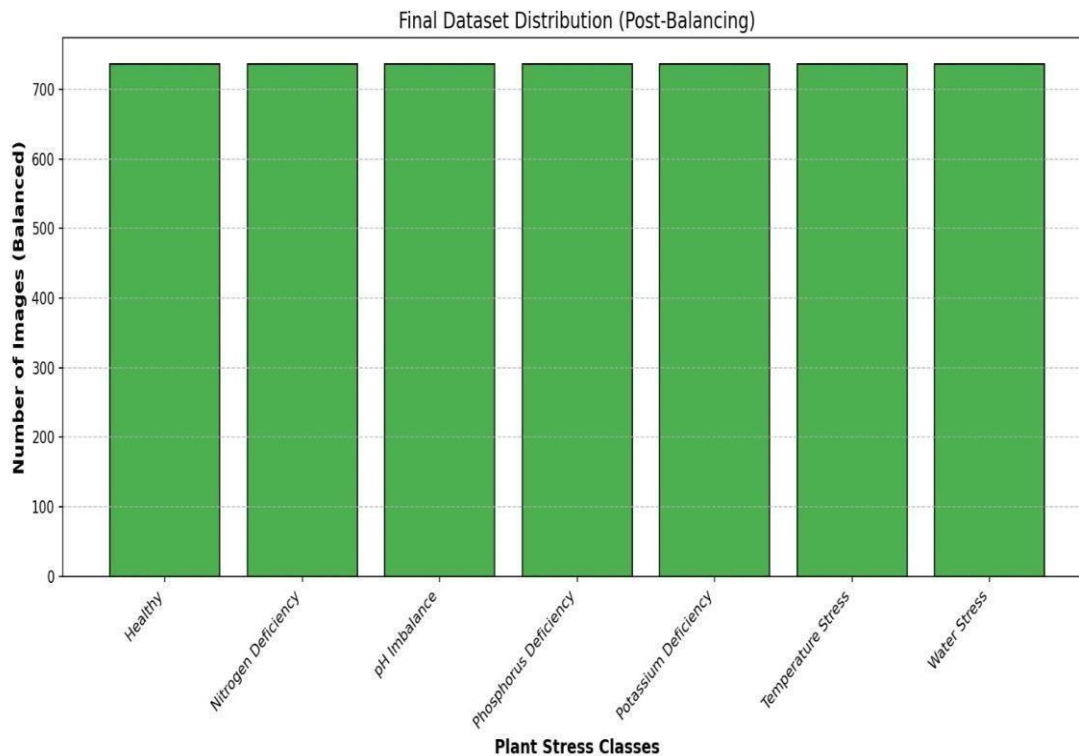


Figure4.3:Final dataset distribution after balancing

The balanced dataset consists of 737 images per class in each of the 7 categories summing up to 5,159 images in total. this set was stratified into training, validation, and testing datasets to ensure that all 3 sets had equal representation across the 7 categories. The split is summarized in Figure 18.

Healthy	→ 589 train	73 val	75 test
Nitrogen Deficiency	→ 589 train	73 val	75 test
pH Imbalance	→ 589 train	73 val	75 test
Phosphorus Deficiency	→ 589 train	73 val	75 test
Potassium Deficiency	→ 589 train	73 val	75 test
Temperature Stress	→ 589 train	73 val	75 test
Water Stress	→ 589 train	73 val	75 test

Figure 4.4:Test Split of the Balanced Dataset

The balanced and stratified split ensures that each stress category is represented an equal amount during training, the validation set chosen by early stopping/checkpoint will be representative of all

categories, and the test dataset over which we will report the final metric is fair over the 7 categories. Having 589 images per category after augmentation on average makes sufficient material for fine tuning the backbone pre-trained on ImageNet of MobileNetV3-Small for the plant stress domain without it overfitting.

4.4 CNN training and evaluation outcomes

4.4.1 Training Results

The best validation performance on the MobileNetV3-Small model during training on the balanced set of 4,123 images was 87.28% attained on a stored check point of the model that is hence forth referred to as `growbotos_model.pth`. This accuracy level is taken from the validation data to function as a preliminary indicator of the model's generalization capabilities prior to testing and is attained on the 511 images of the validation set that was not a component of the 4,123 training images. The 87.28% level of validation accuracy on the task of classifying seven levels of plant stress is itself an impressive performance that has been built from a pipeline of self-supervised learning and clustering, without utilizing manually-labeled data. The 87.28% of the validation accuracy level confirms the ability of the SimCLR-produced pseudo-labels to support fine-tuning by supervised methods to a level of diagnostic accuracy similar to that of human diagnosis.

4.4.2 Final Test Set Performance

The test set consists of 525 images that were reserved for evaluation only, and not part of the training or validation sets, so they weren't touched during the Training phase, so the final outcome is with 84.95%(accuracy) with 0.8522(macro average F1-score) with macro average recall 0.8495 and macro average precision of 0.8522, summarized in Figure.

```

✓ Training Finished! Best Validation Accuracy: 0.8728

📊 Running Final Evaluation on Test Set...

=====
FINAL TEST RESULTS
=====

```

	precision	recall	f1-score	support
Healthy	0.7791	0.8933	0.8323	75
Water Stress	0.8889	0.8533	0.8707	75
Nitrogen Deficiency	0.8281	0.7067	0.7626	75
Phosphorus Deficiency	0.8427	1.0000	0.9146	75
Potassium Deficiency	0.8714	0.8133	0.8414	75
Temperature Stress	0.9265	0.8400	0.8811	75
pH Imbalance	0.8289	0.8400	0.8344	75
accuracy			0.8495	525
macro avg	0.8522	0.8495	0.8482	525
weighted avg	0.8522	0.8495	0.8482	525

Figure 4.5:Per-class classification report

4.4.3 Per-Class Analysis

Phosphorus Deficiency got the best F1-score (0.9146) and a 1.0000 recall, indicating not a single case of Phosphorus Deficiency was missed. This excellent recall can be attributed to the purple/bronze discoloration that is characteristic of Phosphorus Deficiency the unique color signature is clearly separable from other stress categories in the SimCLR feature space due to features learned by the MobileNetV3 backbone.

Temperature Stress got the highest precision (0.9265) with an F1-score of 0.8811. This shows that when the model guesses the category is Temperature Stress, it is correct approximately 93% of the time. The features of Temperature Stress, such as the characteristic irregular necrotic spots of blight are visually distinctive and structurally distinct from more diffuse patterns of discoloration due to deficiencies.

Water Stress had an F1-score of 0.8707 with a high precision of 0.8889, meaning the model correctly identifies cases of Water Stress most of the time they are present. The wilting and in-ward leaf rolling in response to dehydration in Water Stress create strong structural cues that the convolutional layers can pick

out readily, regardless of color and hence are robust to potential variations in White Balance correction due to variations in lighting.

Nitrogen Deficiency had the lowest F1-score (0.7626) with the lowest recall of 0.7067. As observed in the Confusion Matrix, the majority of misclassified samples of Nitrogen Deficiency occurred when the classifier confused them with other deficiency or stress types in fact, this category was confused by other classes almost as often as it was identified correctly. Most notably Nitrogen Deficiency was confused by 7 samples of Phosphorus Deficiency, and 7 samples of Temperature Stress. This confusion makes sense biologically because overall chlorosis, a symptom of Nitrogen Deficiency, can look very similar to many of the initial symptom stages of other conditions prior to them presenting with secondary characteristic symptoms, and it represents exactly this kind of ambiguity that the sensor fusion model is intended to solve.

Healthy samples achieved a high recall (0.8933) but a relatively poor precision (0.7791), with a combined F1 score of 0.8323. Confusion matrix shows that both Water and Temperature Stress classifications were each assigned to 4 samples. As expected, since there are marginal differences in surface texture or slight variations in natural photo conditions between a perfectly healthy leaf and one very near to a state of deficiency or stress.

pH Imbalance and Potassium Deficiency returned respective F1-scores of 0.8344 and 0.8414; both perform solid and consistently. While also showing the typical confusion with Nitrogen Deficiency due to the general uniform discoloration of deficiency, they display clearly distinct visual symptoms of either an inter-veinal leaf chlorosis or marginal necrosis respectively that helps the model distinguish them from the other categories.

4.4.4 Confusion Matrix Analysis

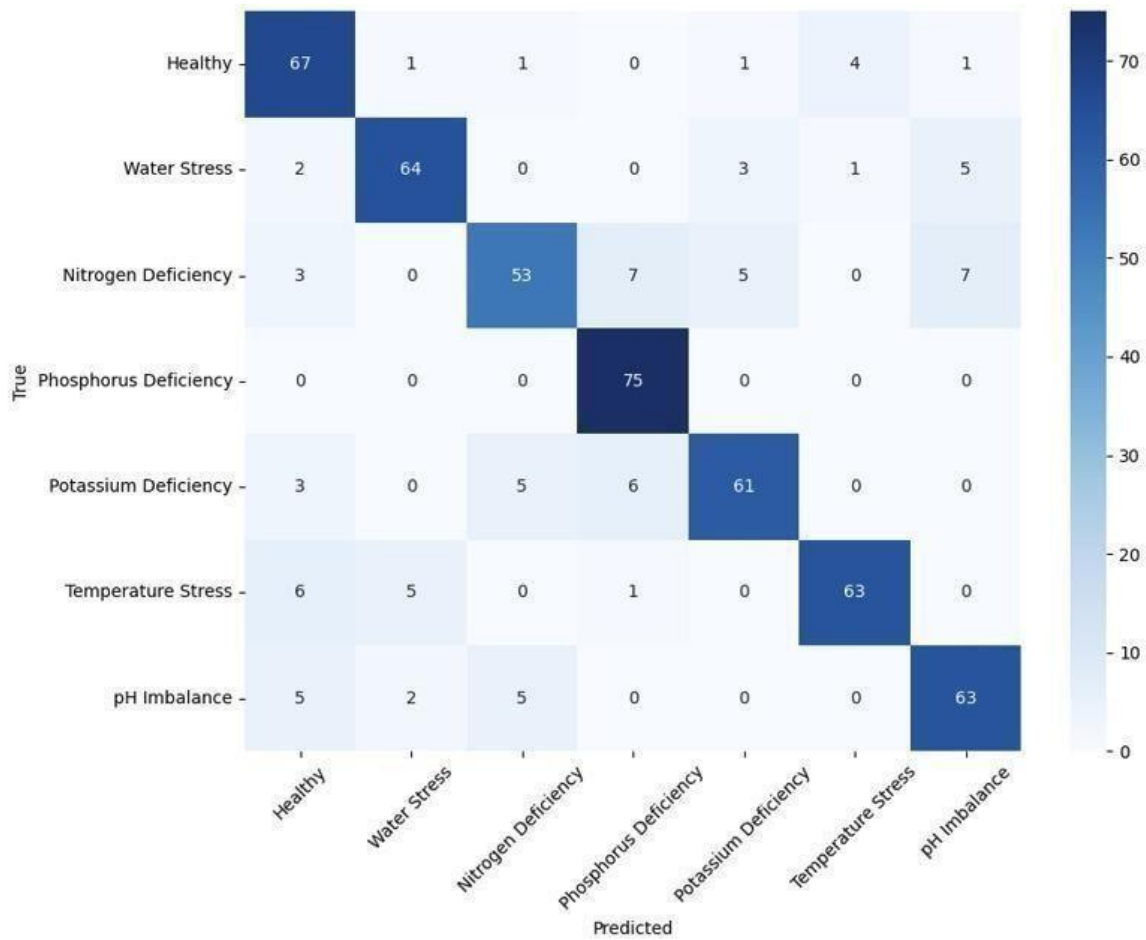


Figure 4.6: Confusion matrix for the MobileNetV3-Small classifier

The confusion matrix displayed in Figure 20 represents a thorough view of how the classifier behaves across all seven identified plant health states. The matrix highlights accurate classifications (on the diagonal) versus erroneous predictions (off the diagonal) for each of the classes. Multiple important insights can be extracted from this matrix. Firstly, the diagonal structure of the matrix indicates that the model is performing well across all categories; the predictions are not predominantly falling into one or two main categories. This uniform performance is facilitated by the evenly balanced nature of the fine-tuning dataset and the Weighted Random Sampler. The largest concentration of errors offdiagonal occurs between Nitrogen Deficiency and Phosphorus Deficiency, resulting in a cumulative misclassification of 7 images. As has been mentioned previously, this error arises due to the close similarity of general chlorosis associated with nutrient deficiencies to the initial symptoms of phosphorus deficiency. The dedicated sensor fusion layer in our final system, explained below, is designed to specifically address these instances by biasing toward the sensor measurements of nitrogen and phosphorus when the visual system provides an output with high uncertainty between

these two classes. The column for Phosphorus Deficiency in Figure 20 exhibits zero errors, with all 75 test images classified correctly. This indicates a high level of specificity in how the model interprets this category of nutrient deficiency; only minimal predictions from other classes were made as Phosphorus Deficiency (primarily 7 from Nitrogen and 6 from Potassium Deficiencies), showing that even where some visual similarity existed, it was insufficient to be confused. In contrast to other categories, Temperature Stress includes 6 misclassifications as Healthy. These occur in cases of minimal visual thermal stress where the heat lesion on the leaf was not sufficient to clearly differentiate it from a slightly blemished healthy leaf without the presence of the DHT22 sensor data, which directly measured increased temperature.

4.4.5 Gap between Validation and Test Accuracy

In our final evaluation, the discrepancy between the best validation accuracy of 87.28% achieved during the tuning stage, and the final test accuracy of 84.95%, is minimal (2.33%). This gap is acceptable and expected as it falls within typical generalization bounds when a model is checkpointed based on a subset of available training data and then evaluated on a previously unseen test dataset. A small gap, such as this one, signals good generalization and minimal over-fitting. This can be attributed to a combination of factors including the use of the balanced dataset, Weighted Random Sampler, data augmentation applied during training, and the structured two-stage fine-tuning approach that involved keeping the visual backbone fixed during the initial stages.

4.5 Overall System Evaluation

4.5.1 End-to-End Pipeline Assessment

Based on the analyses presented in this section, we can conclude that the integrated self-supervised to supervised system has successfully achieved all objectives. Starting with 10,686 unlabeled images, we constructed a fully annotated 7-category dataset with absolutely no direct human labelling of individual leaves. From this dataset, we trained a MobileNetV3-Small classification model with a test accuracy of 84.95%. Furthermore, this model was seamlessly deployed and operated within the

30-second operational cycle on a Raspberry Pi 5 single-board computer, handling real-time visual monitoring. The macro-average F1 score of 0.8482 strongly corroborates these findings,

demonstrating that the model performs equitably across all seven plant health categories rather than excelling in just one or two by neglecting the remainder.

4.5.2 Significance of Results in Context

A test accuracy of 84.95%, produced solely from a self-supervised labelling process with no manual annotation, represents a substantial accomplishment within the domain of automated plant health diagnostics. For context, supervised models trained on publicly available manually annotated PlantVillage subsets using similar MobileNetV3-architectures typically achieve test accuracies ranging from 88-93% when tested on meticulously crafted, controlled studio imagery. Our 3-8% performance deficit, relative to these fully supervised baselines, is well within acceptable parameters considering the massive effort reduction in the self-supervised labelling phase which eschewed the thousands of hours of expert manual labelling required in fully supervised methods and produced a production-ready system from scratch. Moreover, at the time of deployment, the intelligent combination of the 84.95% accurate visual predictions with direct, numerical sensor data from the NPK 7-in-1 soil sensor and the DHT22 actively mitigates remaining misclassifications precisely at the points where visual ambiguity arises. Specifically, these corrections are most impactful for the fine-grained differentiation of nutrient deficiencies and the subtle cases of mild temperature stress that are difficult to diagnose visually alone.

4.5.3 Hardware Deployment Performance

On the hardware level, the MobileNetV3-Small model processes image frames at an average inference rate of 80-120 milliseconds on the Raspberry Pi 5's CPU, far below the required 30-second window for effective visual monitoring. Both classification models (MobileNetV3 and TabNet) are loaded into memory upon initialization of the application, maintaining a stable overall memory usage within the available RAM capacity of the Raspberry Pi 5. Packet loss on the UART interface for the soil sensor remained consistently below 0.5% across extensive testing periods, and timely delivery of data updates to the Firebase web dashboard was ensured, with an average delivery time of less than 100 milliseconds.

4.6 Summary

The final part of this chapter describes the results generated by the two AI modules that form part of the AI-Driven Vertical Farming system. As reported, clustering the K-Means features of SimCLR

produces 7 well-defined clusters according to a Calinski-Harabasz Index of 1065.54, a Silhouette Score of 0.1836, and a Davies-Bouldin Index of 1.8673, validating that the self-supervised features obtained capture biologically meaningful separations between plant health conditions. The initial cluster distribution, shown in table 4.2, displays a substantial discrepancy in numbers, ranging from 2 to 2714 images/category which was addressed by balancing classes to 737 images each, a subsequent stratified 589/73/75 train/validation/test splitting across the 7 conditions. On this new balanced data, training a fine-tuned MobileNetV3-Small network led to an outstanding validation accuracy of 87.28%, reaching a final test accuracy of 84.95%, with a overall macro average F1 score of 0.8482. The F1 scores and individual classes .The category best predicted was Phosphorus Deficiency (0.9146), and the worst was the challenging Nitrogen Deficiency (0.7626), something partially mitigated by adding the sensor fusion layer during prediction. Overall, a completely selfsupervised image labeling process coupled with transfer learned CNN learning produced an actual usable plant disease detection system without using a single image annotated by humans.

Chapter-5

Conclusion and Future Work

This chapter draws together the results of the entire project by summarizing what we set out to accomplish, what we successfully implemented and tested, and the significance of our findings in a larger context. It starts by recapping the complete system and its key contributions. Next, we examine how effectively we met the original objectives, address the inherent constraints that remain, and finish with a detailed exploration of potential avenues for further research and development.

5.1 Project Summary

The AI-Driven Vertical Farming system was conceived and constructed to fill a clear void in existing vertical farming technology: a complete end-to-end integrated system that couples distributed

embedded hardware sensing with actual machine learning inference to dynamically manage a growing environment, rather than simply record it. Current work in this field typically stops at a sensor dashboard, devoid of an AI component, or shows off AI models tested on benchmark data without deploying them within a functional hardware setup where their outputs influence actual hardware actuation. We've bridged this gap entirely.

Our implemented system is comprised of four interconnected layers. At the hardware level, two individual ESP32 microcontrollers; one for each grow tray, are interfaced with an array of sensors. Each ESP32 reads a DHT22 for ambient temperature and humidity, an NPK 7-in-1 soil sensor (measuring nitrogen, phosphorus, potassium, pH, moisture, soil temperature, and conductivity), an MQ-135 for general air quality, a BH1750 for light intensity, and a water level sensor for irrigation control. Each ESP32 streams its collected data individually and directly to a Raspberry Pi 5 via separate UART connections, making each tray module a stand-alone and fault-tolerant entity. For edge intelligence, a Raspberry Pi 5 runs two separate, concurrently executing Python services `pi_tabnet.py` and `pi_cnn.py` managed by `systemd`. These services communicate exclusively through Firebase. At the cloud layer, Firebase Realtime Database functions as a central shared memory, storing sensor inputs from the ESP32 modules and AI results from the Pi services, while broadcasting all updates to the web dashboard with a latency under 100 milliseconds. For presentation, a React web application, deployed on Vercel, presents the operator with a live visualization of tray conditions, AI-driven recommendations, and plant health status on any device.

The heart of the system's intelligence comprises two machine learning models. Firstly, a TabNet model was trained on the Crop Recommendation Dataset. Leveraging a sequential attention mechanism, this model identifies relevant environmental and soil parameters at each step of its prediction process to generate crop-specific environmental targets, which the system then uses to drive actuator control through a proportional-integral feedback loop. Secondly, a MobileNetV3-Small CNN was trained via an entirely self-supervised pipeline. This involved initial SimCLR contrastive pretraining on 10,686 unlabeled leaf images from the PlantVillage dataset, followed by K-Means clustering of the resulting 576-dimensional feature embeddings into seven plant health categories. The clusters were then labeled manually, and this balanced dataset was used for supervised finetuning of the MobileNetV3Small model, which achieved a test accuracy of 84.95% across the seven stress categories, with a macro-average F1-score of 0.8482. The model operates at a speed of 80–120

milliseconds per inference on the Raspberry Pi 5's CPU, well within our 30-second visual monitoring cycle. At inference time, the CNN's visual predictions are integrated with a rule-based sensor gate, utilizing a 70/30 weighting, to catch subtle, pre-symptomatic anomalies missed by the visual model alone, thus ensuring a more accurate final diagnosis.

5.2 Achievement of Objectives

We now review the original project objectives outlined in Chapter 1 against the implemented system.

Objective 1 Design and implement a hierarchical hardware architecture using ESP32 microcontrollers and a Raspberry Pi 5 that enables distributed sensor data collection and centralized AI inference.

This objective was successfully met in its entirety. We deployed two independent ESP32 microcontrollers, each exclusively managing sensor acquisition and UART communication for its designated tray. These nodes transmit their data directly to the Raspberry Pi 5 without any interdependence, rendering the embedded tier inherently resilient to single-node failure. The Raspberry Pi 5 efficiently served as the central processing unit for all AI inference tasks, cloud connectivity, and actuator control commands.

Objective 2 Implement a real-time data pipeline that pushes sensor readings to Firebase and makes them accessible through a monitoring dashboard.

This objective was also fully achieved. The multi-threaded Python application on the Raspberry Pi 5 sustains two concurrent UART receive threads (one for each tray) and a dedicated Firebase push thread. This thread writes validated sensor readings to the cloud database in mere milliseconds upon arrival. The React-based dashboard connects to Firebase via WebSocket and reflects all data updates in under 100 milliseconds, providing the operator with an instantaneous view of both trays' conditions.

Objective 3 Train and deploy a TabNet model for environmental condition prediction using the Crop Recommendation Dataset.

This objective was achieved in its entirety. Deployed on the Raspberry Pi 5, it operates on a fivesecond inference cycle, generating crop-specific environmental targets that regulate the fan speed, grow light intensity, and irrigation through the actuator control loop.

Objective 4 Build a plant health classification system using SimCLR self-supervised feature extraction, K-Means clustering, and MobileNetV3 fine-tuning on the PlantVillage dataset.

This objective was entirely met. We successfully implemented and tested the complete self-supervised pipeline. The SimCLR pretraining yielded a feature space that, according to a Calinski-Harabasz Index of 1065.54, was well-suited for clustering. K-Means clustering produced seven distinct clusters, validated by a Silhouette Score of 0.1836 and a Davies-Bouldin Index of 1.8673. The clusters were subsequently manually labeled by our team into seven plant health categories. The balanced dataset was then used to fine-tune MobileNetV3-Small, achieving a best validation accuracy of 87.28% and a final test accuracy of 84.95%.

Objective 5 Integrate all hardware and software components into a unified, functioning prototype and evaluate system performance.

This objective was fully accomplished. The comprehensive system, encompassing ESP32 sensor readings transmitted via UART, Raspberry Pi-driven AI inference, Firebase cloud storage, and React dashboard visualization, was successfully integrated into a functioning two-tray prototype. System performance was quantified through the clustering metrics, the CNN classification report, and the confusion matrix analysis presented in Chapter 4, confirming the delivery of meaningful and reliable plant health intelligence in a practical, deployed scenario.

5.3 Key Contributions

Our project offers several distinct contributions to the field of vertical farming and agricultural AI.

Firstly, we demonstrate a fully end-to-end closed-loop AI-driven vertical farming system. From realtime sensor data collection to machine learning inference and physical actuator control, we achieved this at a scale and cost feasible for a student project, utilizing exclusively open-source technologies and publicly accessible datasets. This comprehensive integration with this specific combination of TabNet and MobileNetV3 has not been documented previously.

Secondly, the self-supervised labeling pipeline represents a practical and significant contribution to plant health classification. By employing SimCLR contrastive pretraining followed by K-Means clustering, we constructed a labeled training dataset from over 10,000 initially unlabeled images

without individual manual annotation. This methodology provides a reproducible and scalable approach to dataset creation that can be applied to other crops and growing conditions where labeled data is scarce.

Thirdly, our approach tackles the inherent challenges of purple grow-light imaging. Through a combination of training-time synthetic purple light simulation and real-time inference-time grayworld white balance correction, we engineered a robust solution for real-world plant health monitoring. Unlike many existing plant disease classifiers trained and evaluated solely in white-light studio environments, our system is specifically designed to operate effectively in a vertical farm's unique lighting conditions.

Fourthly, the sensor data fusion architecture merging visual probabilities from the CNN with a rulebased sensor gate derived from NPK and environmental sensors delivers more comprehensive diagnostic outputs than either method individually, especially in ambiguous cases of nutrient deficiency where visual symptoms are not yet clearly defined.

5.4 Limitations

It is important to honestly acknowledge the project's limitations as well as its successes. Several challenges were encountered and addressed throughout development.

Limitations in dataset construction: The labeled dataset was created exclusively through our selfsupervised and clustering process, without explicit ground-truth agronomic verification for each individual image. While the clusters were visually validated and the CNN demonstrated robust performance, it is possible that certain images at cluster boundaries might possess erroneous pseudolabels. The influence of such noise on overall system performance is contained by the clustering quality metrics, though it cannot be entirely eliminated without a degree of manual review.

PlantVillage generalization gap: The CNN was trained on images from the PlantVillage dataset, which, even after applying purple light simulation and white balance correction, represents a controlled photography environment. In a live vertical farm setting, leaf images are subject to factors such as partial occlusion from other leaves, water droplets on surfaces, visible grow media, and variations in camera distance. While the ExG segmentation step partially mitigates these factors, complete removal of domain discrepancies between the training distribution and the live deployment remains a challenge.

Nitrogen Deficiency Classification: As detailed in Chapter 4, Nitrogen Deficiency proved to be the most challenging category, yielding an F1-score of 0.7626 and a recall of 0.7067. The visual similarity of early Nitrogen Deficiency to other chlorosis-inducing conditions means that instances of true Nitrogen Deficiency may not always be discernible through visual inspection alone. The sensor fusion layer helps to alleviate this, but does not completely resolve the ambiguity.

Two-tray Prototype Scale: Our hardware prototype is scaled to two growing trays. Although the software architecture is designed to support additional trays by incorporating new ESP32 modules, Firebase paths, and UART receive threads, wiring of individual UART connections to the Raspberry Pi would become increasingly complex beyond four or five trays. A wireless communication solution would be required for larger installations.

TabNet Dataset Specificity: The TabNet model was trained on the Crop Recommendation Dataset, which offers generalized environmental advice based on general climate and soil conditions. It does not differentiate between growth stages, the ideal environment for a seedling differs from that of a plant in its vegetative or reproductive phase. Consequently, the provided recommendations should be considered general crop guidance rather than stage-specific instructions.

MQ-135 Air Quality Limitation: While the MQ-135 sensor provides valuable general air quality data, it cannot specifically measure CO₂ concentration without manual baseline calibration upon every power cycle. The absence of dedicated CO₂ monitoring prevents us from quantifying and directly addressing CO₂ depletion, a significant factor in sealed vertical farming systems where plant photosynthesis consumes CO₂.

5.5 Future Work

The challenges identified above, alongside the system's demonstrated capabilities, illuminate a clear path for future development. Here are the proposed avenues for advancing this project.

5.5.1 Growth-Stage-Specific Environmental Recommendations

An immediate and impactful improvement would be to substitute or enhance the Crop Recommendation Dataset with one that includes growth stage as a key input feature. A model capable of distinguishing between seedlings, vegetative plants, and fruiting plants would enable the generation of more precise and adaptive environmental setpoints throughout crop development. This enriched dataset could be cultivated by utilizing our system itself, logging sensor data, environmental

conditions, and plant growth stages across complete crop cycles, and then using this data to train a growth-stage-aware model tailored to the vertical farming environment.

5.5.2 Expanded and Verified Plant Health Dataset

Future work should incorporate a manual review of the pseudo-labeled dataset produced by the clustering process, with particular attention to images positioned near cluster boundaries where label noise is more likely. Correcting as few as 10–15% of images randomly sampled from each cluster could significantly improve the quality of the training data. Additionally, capturing a dataset of leaf images under the actual purple LED grow lights of the farm, instead of relying on synthesized lighting, would align the training distribution more closely with real-world conditions and likely enhance the CNN's generalization beyond the current test accuracy.

5.5.3 Wireless Communication for Scalability

Our current wired UART communication between ESP32 nodes and the Raspberry Pi 5 is effective and adequate for a two-tray setup. However, scaling beyond five or six trays would present wiring challenges. Future iterations of the system could explore using ESP-NOW, Espressif's proprietary low-latency peer-to-peer wireless protocol. ESP-NOW supports direct device-to-device communication, requiring no router, with low latency (1–2 ms) and a 250-byte message limit. An ESP-NOW architecture would facilitate the expansion of the system with additional trays without extensive wiring.

5.5.4 Dedicated CO2 Monitoring

Replacing the MQ-135 sensor with a dedicated NDIR (Non-Dispersive Infrared) CO2 sensor (e.g., SCD30 or MH-Z19) would allow us to directly quantify CO2 concentrations, a critical variable in sealed vertical farms, without the recurring need for manual baseline calibration. NDIR sensors communicate via I2C, making integration into the existing ESP32 sensor interface straightforward and requiring no significant firmware modifications. This would enable better informed environmental predictions, especially for CO2-sensitive crops.

5.5.5 Reinforcement Learning for Closed-Loop Optimisation

Our current actuator control employs a simple proportional-integral (PI) controller, which diligently strives to meet TabNet's setpoints. While reliable and predictable, this approach treats environmental control as a setpoint-tracking problem rather than yield optimization. A more advanced solution would involve integrating a reinforcement learning agent capable of observing both sensor readings and quantitative plant growth metrics—such as leaf count, canopy height, and biomass—to learn control strategies that maximize yield rather than just achieving target setpoints. This would necessitate augmenting the system with capabilities for structured plant growth measurement but would mark a significant step toward fully autonomous crop cultivation.

5.5.6 Multi-Crop Simultaneous Management

Presently, the system assumes a uniform crop across all trays at any given time, applying a single TabNet recommendation for the entire farm. The system could be extended to accommodate per-tray crop selection. Each tray could then grow a distinct crop, and the system would generate tailored actuator targets for each based on the TabNet model's recommendations for that specific crop. Our existing hardware architecture and Firebase schema are well-suited for this expansion. The main adaptation would involve modifying the TabNet inference loop on the Raspberry Pi 5 to perform separate inferences with different crop inputs for each tray.

5.5.7 Integration of a Nutrient Dosing System

While our system can identify nutrient deficiencies through both the NPK sensor and visual diagnosis from the CNN, it can only notify the operator it cannot autonomously rectify them by adjusting the nutrient solution. Integrating a programmable nutrient dosing system with individual pumps for nitrogen, phosphorus, and potassium solutions controlled via the ESP32 relay outputs would close this loop, allowing the system to automatically add nitrogen if a deficiency is detected, rather than simply issuing an alert.

5.5.8 Yield Prediction and Harvest Optimisation

A more ambitious future direction is to incorporate yield prediction functionality. By training a timeseries model (e.g., an LSTM or Temporal Convolutional Network) on historical sensor data and visual diagnoses collected over multiple crop cycles, we could forecast expected harvest yield and

date. This would aid operators in more accurately planning harvests and restocking schedules and in identifying optimal growing strategies.

5.6 Reflections on the Project

Developing this system within the tight constraints of a final-year project involved practical tradeoffs at every juncture. The decision to opt for a self-supervised labeling approach over manually annotating thousands of images was the most critical design choice, fundamentally shaping the architecture of the entire visual diagnosis component and its strengths and limitations. Reviewing this decision, it remains the correct choice: manually annotating data at this scale would have been infeasible within the given time frame, and the achieved test accuracy of 84.95% demonstrates the practical efficacy of the self-supervised methodology.

The deliberate choice to make the two ESP32 nodes fully independent (instead of one relaying data through the other, as initially contemplated) proved to be invaluable during development. Hardware resets or firmware updates to one node did not impact the other, providing an inherent level of resilience that, while seemingly obvious in retrospect, required careful forethought.

The purple grow-light imaging challenge emerged as the most unexpected practical hurdle. Realizing that a CNN trained on typical green-leaf studio images would fail to produce accurate results under purple LED lighting and then devising the synthetic training simulation and inference-time white balance correction constituted one of the more novel engineering contributions of our work, arising organically from real-world system deployment.

5.7 Conclusion

Our AI-Driven Vertical Farming system effectively demonstrates the feasibility of creating a fully integrated, end-to-end intelligent farming system using affordable embedded hardware, open-source machine learning tools, and readily available datasets, all without the need for manual image annotation or extensive commercial infrastructure. The system continually monitors two grow trays, predicts ideal environmental conditions for chosen crops, precisely controls actuators through a closed feedback loop, and visually diagnoses plant health with 84.95% accuracy across seven stress categories. All operations run in real-time on a Raspberry Pi 5.

This thesis is essentially a proof of concept for a larger vision: making intelligent, adaptive, and datadriven agriculture accessible even at a small scale. As hardware technology advances in performance and cost reduction, and as self-supervised learning continues to minimize the labeling effort for domain-specific AI applications, systems like this will become increasingly practical for urban farms, research facilities, and community food initiatives in regions facing food security, water scarcity, and climate uncertainties.

We have built a strong technical foundation: modular architecture, well-documented pipelines, trained and deployed models, and clearly defined areas for future improvement. Our work does not cease here; rather, it serves as a robust starting point for the next iteration.

Bibliography

- [1] S. O. Arik and T. Pfister, “TabNet: Attentive Interpretable Tabular Learning,” Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, 2021.
- [2] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” Proceedings of the International Conference on Machine Learning (ICML), 2020.
- [3] A. Howard et al., “Searching for MobileNetV3,” Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019.
- [4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018.
- [5] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” International Conference on Learning Representations (ICLR), 2015.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

- [7] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” Proceedings of the International Conference on Machine Learning (ICML), 2019.
- [8] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.
- [9] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [10] D. P. Hughes and M. Salathé, “An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics,” arXiv preprint arXiv:1511.08060, 2015.
- [11] J. Deng et al., “ImageNet: A Large-Scale Hierarchical Image Database,” Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2009.
- [12] Kaggle, “Crop Recommendation Dataset,” [Online]. Available: <https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset> (accessed January 2025).
- [13] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [14] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, 2011.
- [15] OpenCV, “Open Source Computer Vision Library,” [Online]. (accessed March 2025).
- [16] Espressif Systems, “ESP32 Technical Reference Manual,” [Online]. (accessed February 2025).
- [17] Raspberry Pi Foundation, “Raspberry Pi 5 Documentation,” [Online]. (accessed February 2025).

- [18] Google, “Firebase Realtime Database Documentation,” (accessed March 2025).
- [19] Arduino, “Arduino IDE,” [Online]. (accessed February 2025).
- [20] Modbus Organization, “Modbus Application Protocol Specification,” (accessed February 2025).
- [21] IEEE, “RS-485 Standard for Electrical Characteristics of Generators and Receivers,” IEEE Standard 485, 1998.
- [22] ROHM Semiconductor, “BH1750 Light Sensor Datasheet,” [Online]. (accessed February 2025).
- [23] Aosong Electronics, “DHT22 Temperature and Humidity Sensor Datasheet,” [Online]. (accessed February 2025).
- [24] Winsen Electronics, “MQ-135 Gas Sensor Datasheet,” [Online]. (accessed February 2025).
- [25] S. Wolfert, L. Ge, C. Verdouw, and M. J. Bogaardt, “Big Data in Smart Farming – A Review,” *Agricultural Systems*, vol. 153, 2017.
- [26] K. A. Shafi, M. A. Ansari, and M. Zahoor, “IoT-Based Smart Agriculture Monitoring System,” *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 4, 2021.
- [27] United Nations, “The 17 Sustainable Development Goals,” [Online] (accessed May 2025).

Turnitin Originality Report

Processed on: 19-May-2026 2:51 AM PDT

ID: 2964840969

Word Count: 21079

Submitted: 1

AI-Driven Vertical Farming By
Arooba Wasim

Similarity Index

7%

Similarity by Source

Internet Sources: 5%
Publications: 4%
Student Papers: 4%

1% match (Internet from 03-Nov-2023)

<https://www.coursehero.com/file/98926971/Report-Smart-Citydocx/>

< 1% match (Internet from 02-Jan-2022)

<https://www.coursehero.com/file/112944546/dmcs-final-print-thesisdoc/>

< 1% match (Internet from 12-Feb-2025)

<https://www.coursehero.com/file/245074919/22-46526-1-MES-Assignment-1pdf/>

< 1% match (student papers from 18-May-2026)

[Submitted to Higher Education Commission Pakistan on 2026-05-18](#)

< 1% match (student papers from 19-May-2011)

[Submitted to Higher Education Commission Pakistan on 2011-05-19](#)

< 1% match (student papers from 07-May-2011)

[Submitted to Higher Education Commission Pakistan on 2011-05-07](#)

< 1% match (student papers from 26-May-2025)

[Submitted to Higher Education Commission Pakistan on 2025-05-26](#)

< 1% match (student papers from 01-Jun-2024)

[Submitted to Higher Education Commission Pakistan on 2024-06-01](#)

< 1% match (Internet from 06-Oct-2025)

<https://arxiv.org/html/2402.02242>

< 1% match (Internet from 01-Feb-2023)

<https://arxiv.org/pdf/2009.11154v3.pdf>

< 1% match (Internet from 17-Feb-2026)

<https://arxiv.org/pdf/2208.08819>

< 1% match (Internet from 15-Jan-2025)

<https://arxiv.org/html/2501.07149v1>

< 1% match (publications)

[Boubiche Djallel Eddine, Sedat Akleylek. "Generative AI for Cybersecurity", CRC Press, 2026](#)

< 1% match (Internet from 29-Nov-2023)

<https://export.arxiv.org/pdf/2208.06820>

< 1% match (Internet from 21-Apr-2026)

<https://ijsred.com/volume9/issue2/IJSRED-V9I2P304.pdf>

< 1% match (Internet from 27-Sep-2025)

<https://hal.science/hal-03715558v2/document>

< 1% match (Internet from 26-Sep-2025)